CHAPITRE 1 : SYSTEME DE NUMERATION

Objectifs :

- Connaître les différents systèmes de numération,
- Savoir convertir un nombre décimal dans différentes bases,
- Savoir convertir un nombre d'une base à une autre,
- Savoir effectuer des opérations arithmétiques binaires et dans différentes bases.

Introduction

Depuis la nuit des temps, l'Homme a eu besoin de **compter** et de **calculer**. Selon les époques ou civilisations, divers **systèmes de numération** ont été mis en place puis abandonnés.
A l'heure actuelle, nous utilisons **le système de numération décimal**. Ce système s'est imposé en Europe à partir du 10ème siècle. Aujourd'hui le système décimal est quasiment universel. Cependant il est mal adapté au codage des informations pour un ordinateur ; on lui préfère le **système binaire**.

I. LES BASES DE NUMERATION

I.1. Chiffre (ou symbole) et nombre

- Dans toute numération, il faut distinguer les **chiffres** (ou symboles) et les **nombres**.
- Un **nombre** est le résultat du comptage d'un ensemble d'objets, d'animaux, de personnes ... Un **nombre** s'écrit avec un ou plusieurs **chiffres** (ou symboles).
- Dans le système décimal, les **nombres** sont écrits à partir des **dix chiffres** : **0, 1, 2, 3, 4, 5, 6, 7, 8 et 9.**

Exemples :
- une année comporte douze mois : 12 est un *nombre* comportant deux *chiffres* **:** 1 et 2.
- l'année 2017 est un nombre comportant 4 chiffres (2, 0, 1 et 7).
- une semaine contient 7 jours : 7 est un *nombre* qui s'écrit avec un seul *chiffre* (7).

I.2. Numération additive et numération de position

Différentes techniques de numération ont dû être mises au point afin de dénombrer des ensembles. Il existe deux grandes techniques de numération : **la numération additive** et **la numération de position (ou positionnelle)**.

- Dans la **numération additive**, chaque symbole a une valeur propre et il suffit d'ajouter les valeurs des symboles pour obtenir le nombre.

  Autre définition :

  Un système de numération est dit additif lorsqu'il utilise des signes qui représentent chacun une valeur et lorsque, pour connaître la valeur du nombre ainsi représenté, il faut additionner les valeurs des différents signes.

  Les plus anciens systèmes de numérations connus sont additifs. Ils ne le sont parfois qu'en partie, combinant ce système avec un autre. Ils ont été supplantés par l'écriture décimale positionnelle moderne qui permet d'effectuer plus facilement la grande majorité des calculs.

- Dans la **numération de position**, la valeur des symboles change en fonction de leur place dans le nombre.

II. LES SYSTEMES DE NUMERATION

Pour écrire un nombre **N** dans une *base b*, on décompose ce nombre dans l'ordre des puissances décroissantes de la base. Le nombre **N** s'écrit de façon unique sous la forme :
$\mathbf{N = a_n.B^n + ... + a_2.B^2 + a_1.B^1 + a_0.B^0}$
Avec : **n** : un entier naturel
B : la base de numération
$\mathbf{a_i}$: les chiffres associés à la base tels que $0 \leq a_i < b$.
Comme ces systèmes de numération sont *positionnelles*, on peut écrire la suite des chiffres sous la forme : $\mathbf{N = (a_n \ldots a_2 a_1 a_0)_b}$. La base b étant notée en indice et codée en décimal.

Les bases utilisées en électronique numérique, que ce soit pour l'arithmétique ou pour représenter de manière compacte des nombres binaires de grande longueur, sont les bases
- **2** (binaire)
- **8** (octal)
- **10** (décimal)
- **16** (hexadécimal), noté aussi **H**

II.1. Numération décimale

Ce système de numération, usuel dans la vie quotidienne, dispose de dix symboles (les chiffres) : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
On travaille alors en **base 10**.

Exemple : $(2018)_{10} = 2.10^3 + 0.10^2 + 1.10^1 + 8.10^0 = 2018$

II.2. Numération octale

Ce système utilise 8 symboles : 0, 1, 2, 3, 4, 5, 6, 7. Il n'est plus employé aujourd'hui, puisqu'il servait au codage des nombres dans les ordinateurs de première génération.

Exemple : $(2018)_8 = 2.8^3 + 0.8^2 + 1.8^1 + 7.8^0 =$

II.3. Numération binaire

La numération en base deux (ou numération binaire) utilise deux symboles 0 et 1. Cette base est très commode pour distinguer les deux états logiques fondamentaux.

Exemple : $(101)_2 = 1.2^2 + 0.2^1 + 1.2^0 = (5)_{10}$

Un nombre à *n* chiffres en base deux distingue 2^n états.
Un état binaire est appelé *bit* (contraction de *binary digit*). Un bit prend les valeurs 0 ou 1.
Les puissances successives de 2 (1, 2, 4, 8, 16, 32, 64, 128, 256,...) sont appelées *poids binaires*.
En général, le poids du bit de rang n est 2^n (*on commence toujours au rang 0*).
Le bit de poids le plus fort est appelé MSB (**M**ost **S**ignificant **B**it).
Le bit de poids le plus faible est appelé LSB (**L**ess **S**ignificant **B**it).

En groupant les bits on peut stocker des valeurs différentes de 0 ou 1:

• Un groupe de 4 bits est appelé "quartet". Il permet de stocker 2^4 valeurs différentes, soit 16 valeurs.
• Un groupe de 8 bits est appelé "octet". Il permet de stocker 2^8 valeurs différentes, c'est à dire 256 valeurs.
• 16 bits forment un "mot". Il permet de stocker 2^{16} valeurs, soit 65 536 valeurs.
• 32 bits forment un "mot double". On peut stocker 2^{32} valeurs, c'est-à-dire plus de 4 milliards.

Taille usuelle des mots binaires :

Taille du mot	Valeurs en binaire
8 bits	0 - 255
16 bits	0 - 65535 (64 K)
32 bits	0 - 4294967295 (4096 M)

L'unité de mesure de la mémoire est l'octet. Un groupe de 1024 octets est appelé kilo-octet, bien que le kilo corresponde en physique à 1000. L'abréviation de kilo-octet est ko (avec un petit k et un petit o).
On trouve d'autres multiples, à savoir le Méga-octet (Mo qui fait 1024 ko), le Giga-octet (Go qui fait 1024 Mo) et le Tera-octet (To, égal à 1024 Go).

En anglais un octet est appelé "byte" (ce qui se dit "baïte"). Un kilo-octet se dit donc "kilobyte". Et l'abréviation devient kB (avec un petit k et un grand B).

II.4. Numération hexadécimale

Le développement des systèmes microprogrammés (mini- et micro-ordinateurs) a favorisé l'utilisation de ce code. Il comporte 16 symboles : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.
Un **quartet**, ou digit hexadécimal, évolue entre 0 et 15 (en base 10) soit 0 et F en hexadécimal. L'assemblage de 2 quartets forme un **octet** qui varie de 0 à 255 (en décimal).
Pour indiquer la base 16, on peut la noter en indice suivant la manière générale. Mais dans la pratique on utilise une autre notation. On place le caractère $ (dollar) devant le nombre ou H derrière.

Exemple : $(AA)_{16}$ = AAH = $AA = A.161+A.160 = 10.16+10.1 = $(170)_{10}$

Il existe d'autres systèmes de numération tels que la numération duodécimale (ase 12), vicésimale (base 20) et sexagésimal (base 60)

III. CHANGEMENT DE BASE - CONVERSION

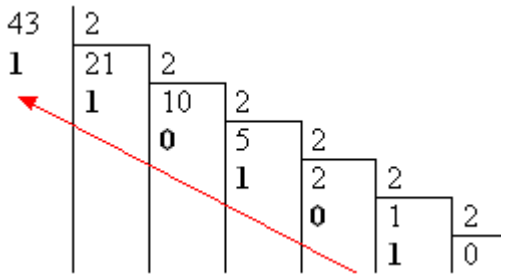
III.1. Conversion décimal vers un système en base B ≠ 10

Il existe plusieurs moyens d'effectuer une telle conversion :
- par soustractions successives des poids binaires dans la différence de l'étape précédente ;
- par divisions successives par B du dividende de l'étape précédente. Les restes correspondants sont les bits consécutifs. C'est terminé au premier dividende nul (que l'on de compte pas).

Méthode de décomposition d'un nombre dans une base par la méthode des divisions successives

La méthode de décomposition par divisions successives consiste à diviser le nombre plusieurs fois (si nécessaire) dans la base choisie jusqu'à obtenir un quotient nul.
Les *restes successifs* des divisions, pris dans leur *ordre inverse*, forment le nombre désiré.

Exercice d'application : soit à convertir $N = (43)_{10}$ en base 2



$N = (101011)_2$

Exemples :

a) Traduire le nombre $(3504)_{10}$ en base 16 puis en base 3.
b) Traduire de même, le nombre $(2018)_{10}$ en base 8 puis en base 2.

III.2. Toutes les conversions vers le décimal

Dans tous les cas, il n'y a rien de particulier à ajouter. Le principe de conversion est directement attaché à la manière dont on écrit un nombre dans une base donnée.
$(N)_B = a_{n-1}.B^{n-1} + \ldots + a_0.B_0$ où B est codé en décimal
La conversion est réalisée automatiquement dans la mesure où le résultat est écrit directement dans la base dix.

III.3. Conversion du binaire vers l'hexadécimal

Un nombre hexadécimal est "découpable" en quartets facilement codables en binaire. Donc, pour convertir du binaire en hexadécimal, on divise le nombre binaire en "tranches de quatre" en partant de la droite. Chacun des "paquets" est ensuite converti en hexadécimal. Cette méthode revient à fractionner en décompositions successives.

Exemple : $(110101110101)_2 = (1101\ 0111\ 0101)_2 = D75H$

Explication : la mise en paquet revient à effectuer une série de factorisations partielles de la base de destination. Ici c'est $16 = 2^4$. Les résidus constituent les chiffres de la conversion. Dans l'exemple précédent, cela donne :
$(110101110101)_2 = 1.2^{11} + 1.2^{10} + 0.2^9 + 1.2^8 + 0.2^7 + 1.2^6 + 1.2^5 + 1.2^4 + 0.2^3 + 1.2^2 + 0.2^1 + 1.2^0$
$= \{1.2^3+1.2^2+0.2^1+1.2^0\}.(2^4)^2 + \{0.2^3+1.2^2+1.2^1+1.2^0\}.2^4 + \{0.2^3+1.2^2+0.2^1+1.2^0\}$
$= 13.16^2 + 7.16^1 + 5.16^0$
$= D75H$

III.4. Conversion de l'hexadécimal vers le binaire

C'est le processus directement inverse, on écrit chaque quartet sur 4 bits en complétant éventuellement avec des zéros.

Exemple : $BC34H = (1011[B]\ 1100[C]\ 0011[3]\ 0100[4])_2 = (1011\ 1100\ 0011\ 0100)_2$.
Exercice d'application : Convertir en binaire EE10AH, AA098CH et IEEE2009H.

➔**Utilité du système hexadécimal**

La facilité avec laquelle se font les conversions entre les systèmes binaire et hexadécimal explique pourquoi le système hexadécimal est devenu une façon abrégée d'exprimer de grands nombres binaires. Dans un ordinateur, il n'est pas rare de retrouver des nombres binaires ayant jusqu'à 64 bits de longueur. Ces nombres binaires, comme nous le verrons, ne sont pas toujours des valeurs numériques, mais peuvent correspondre à un certain code représentant des renseignements non numériques. Dans un ordinateur, un nombre binaire peut être :
1) un vrai nombre ;
2) un nombre correspondant à un emplacement (adresse) en mémoire ;
3) un code d'instruction ;
4) un code correspondant à un caractère alphabétique ou non numérique ; ou
5) un groupe de bits indiquant la situation dans laquelle se trouvent des dispositifs internes et externes de l'ordinateur.
Quand on doit travailler avec beaucoup de nombres binaires très longs, il est plus commode et plus rapide d'écrire ces nombres en hexadécimal plutôt qu'en binaire. Toutefois, ne perdez pas de vue que les circuits et les systèmes numériques fonctionnent exclusivement en binaire et que c'est par pur souci de commodité pour les opérateurs qu'on emploie la notation hexadécimale.

III.5. <u>Conversion du binaire vers l'octal</u>

On reprend les mêmes principes, sachant que $8 = 2^3$ (en factorisant $8 = 2^3$).
Le principal avantage du système de numération octal réside dans la facilité avec laquelle il est possible de passer d'un nombre octal à un nombre binaire. Cette conversion s'effectue en transformant chaque chiffre du nombre octal en son équivalent binaire de trois chiffres. Voyez dans le tableau ci-dessous les huit symboles octaux exprimés en binaire.

Symbole octal	0	1	2	3	4	5	6	7
Equivalent binaire	000	001	010	011	100	101	110	111

Au moyen de ce tableau, tout nombre octal est converti en binaire par la transformation de chacun des chiffres. Par exemple, la conversion de $(472)_8$ va comme suit :

$$\begin{array}{ccccc} (& 4 & 7 & 2 &)_8 \\ & 100 & 111 & 010 & \end{array}$$

Donc le nombre octal $(472)_8$ est équivalent au nombre binaire 100111010.

III.6. Conversion de l'octal vers le binaire

La conversion d'un nombre binaire en un nombre octal est tout simplement l'inverse de la marche à suivre précédente. Il suffit de faire avec le nombre binaire des groupes de trois bits en partant du chiffre de poids le plus faible, puis de convertir ces triplets en leur équivalent octal (voir tableau ci-dessus). À titre d'illustration, convertissons $(100111010)_2$ en octal.

100 111 010
(4 7 2)$_8$

Parfois, il arrivera que le nombre binaire ne forme pas un nombre juste de groupes de trois. Dans ce cas, on pourra ajouter un ou deux zéros à gauche du bit de poids le plus fort pour former le dernier triplet (si on lit de droite à gauche). Voici une illustration de ceci avec le nombre binaire 11010110.

011 010 110
(3 2 6)$_8$

Notez l'ajout d'un zéro à gauche du bit de poids le plus fort pour obtenir un nombre juste de triplets.

IV. ARITHMETIQUE DANS UN SYSTEME DE NUMERATION DE BASE *B*

Les opérations se font exactement comme dans le système décimal sauf qu'ici le rôle de la dizaine est joué par la base B

➔ Opérations dans la représentation binaire naturelle

Table d'addition binaire		
Opérations	Résultat	Retenue
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1

Table de soustraction binaire		
Opérations	Résultat	Retenue
0 – 0	0	0
0 – 1	1	1
1 – 0	1	0
1 – 1	0	0

Table de multiplication binaire		
Opérations	Résultat	Retenue
0 x 0	0	0
0 x 1	0	0
1 x 0	0	0
1 x 1	1	0

Exemple :

➔ Pour écrire un nombre rationnel dans un système de base B, on transcrit numérateur et dénominateur en base B et on pose la division.

Exemple : Ecrire les décimaux 5/8 et 3/5 en base 2

2-Transcodage d'un nombre réel

Soit un nombre réel N à transcoder en base B.

N, qui s'écrira $N = (a_n a_1 a_0 , a_{-1} a_{-2} a_{-3} \ldots)_B$ se décompose sous la forme $N = a_n B^n + \ldots + a_1 B^1 + a_0 B^0 + a_{-1} B^{-1} + a_{-2} B^{-2} + a_{-3} B^3 + \ldots$

Pour convertir un nombre décimal à virgule dans une base **B** quelconque, il faut :
 Convertir la partie entière en effectuant des divisions successives par **B** (comme nous l'avons vu précédemment).
 Convertir la partie fractionnaire en effectuent des multiplications successives par **B** et en conservant à chaque fois le chiffre devenant entier.

Autrement dit :

La **partie entière** se met sous la forme :
$Ent[N] = a_n B^n + \ldots + a_1 B^1 + a_0 B^0 = (a_n B^{n-1} + \ldots + a_1 B^0) B + a_0 B^0$
Les coefficients a_1, a_2, a_3 , ... sont obtenus par **divisions successives par B**.

La **partie fractionnaire** pouvant se mettre sous la forme :

$Fract[N] = a_{-1}B^{-1} + a_{-2}B^{-2} + a_{-3}B^{-3} + \ldots$
Les coefficients a_{-1}, a_{-2}, a_{-3}, ... sont obtenus par **multiplications successives par B**.
$B.Frac[N] = a_{-1}B^{0} + a_{-2}B^{-1} + a_{-3}B^{-2} + \ldots$

Exemple : soit à convertir $(0,64)_{10}$ en base 2

0,64	0,28	0,56	0,12	0,24
2	2	2	2	2
1,28	**0**,56	**1**,12	**0**,24	**0**,48

Recherche de la partie fractionnaire
par **multiplications successives par B=2**
Résultat : $(0,64)_{10} = (0,10100)_2$
Note : pour des nombres de taille plus importante, il sera plus rapide de passer par la base B=16

La conversion d'une base B vers la base 10 se fait, comme pour les entiers (vu plus haut), en attribuant le poids de chaque chiffre.

Exemples :

$$(243,64)_{10} = 2\times100 + 4\times10 + 3\times1 + 6\times0,1 + 4\times0,01 \quad \text{(pour mémoire!)}$$
$$(101011,101)_2 = 1\times32 + 0\times16 + 1\times8 + 0\times4 + 1\times2 + 1\times1 + 1\times0,5 + 0\times0,25 + 1\times0,125$$
$$= (43,625)_{10}$$
$$(2B,A)_{16} = 2\times16 + B\times1 + A\times0,0625 = 2\times16 + 11\times1 + 10\times0,0625 = (43,625)_{10}$$

Convertir le nombre (68,625) en base 2 puis en base 8

V. REPRESENTATION DES NOMBRES AVEC LEUR SIGNE

La représentation des nombres non signés (donc positifs) dans différentes bases a été vue plus haut. Qu'en est-il des nombres négatifs ?

V.1. Représentation binaire des nombres avec leur signe

Dans une représentation codée des nombres, seule est prise en compte leur valeur absolue laquelle est représentée par un ensemble d'éléments binaires. Si l'on veut mettre en évidence le signe d'un nombre, il faut utiliser une information complémentaire. La logique voudrait que l'on rajoute un signe devant. Oui mais comment le faire dans un processeur ?

Il faut donc trouver un codage pour ces nombres (le plus efficace possible).

Une des méthodes (la plus simple) consiste à ajouter un bit au module du nombre pour représenter le signe. Habituellement on associe 0 au signe "+" et 1 au signe "-", au bit que l'on place à gauche du module ; dans ces conditions le nombre zéro (0) a deux représentations. Il est évident que cette méthode ne pose aucune difficulté lorsque l'on a à effectuer des multiplications

	Nombre binaire	
Nombres décimaux	Signe	Valeur
.	.	.
.	.	.
.	.	.
+4	0	1 0 0
+3	0	0 1 1
+2	0	0 1 0
+1	0	0 0 1
+0	0	0 0 0
-0	1	0 0 0
-1	1	0 0 1
-2	1	0 1 0
-3	1	0 1 1
-4	1	1 0 0
.	.	.
.	.	.
.	.	.

ou divisions. Par contre dans le cas d'additions de nombres négatifs (ou soustractions) il faut d'abord comparer les valeurs absolues des nombres à additionner pour connaître le signe du résultat.

Deux autres façons de coder un nombre négatif est l'utilisation du **complément restreint (CR)** et du **complément vrai (CV).**

- Le **complément restreint (CR)** d'un nombre s'obtient en partant du nombre positif et en cherchant pour chaque chiffre, le complément pour atteindre b-1 (b étant la base).

Dans ce système, comme précédemment, la valeur « 0 » a également deux représentations : « +0 » et « -0 »

- **Complément vrai (CV)**

Ce système permet de pallier au défaut de la double représentation de « 0 ». Cette méthode consiste à réaliser un complément restreint d'u nombre N, puis d'ajouter 1 au résultat. Autrement dit : **CV(N) = CR(N) + 1**

➔ Représentation des nombres négatifs : Code complément à 2

V.2. Représentation des nombres fractionnaires

Un nombre fractionnaire comporte deux parties :

- ✓ une valeur entière,
- ✓ suivie d'une valeur fractionnaire.

Les deux parties sont séparées par une virgule qui se place à droite du chiffre le moins significatif de la partie entière (chiffre de poids unité).

On considère deux types de représentations. Dans chacune d'elles les performances sont définies par :

- La RESOLUTION qui est la différence entre deux nombres consécutifs,

- La DYNAMIQUE qui est la différence entre le nombre le plus grand et le plus petit que l'on peut traiter dans un format donné.

a) Représentation en virgule fixe

Un tel nombre est constitué d'un bloc d'éléments binaires comme un entier. Ce bloc représente un nombre fractionnaire si l'on connaît la place de la virgule (qui est positionnée par l'opérateur en fonction des besoins). Ce positionnement doit rester fixe pendant toute la conduite du calcul. Ainsi la virgule délimite deux zones : la partie entière et la partie fractionnaire du nombre.
Les éléments situés à droite de la virgule sont associés à des puissances négatives de la base B.

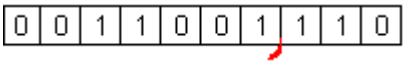
Exemple en complément à 2

0110,11 est un nombre positif qui vaut :
$1.2^2 + 1.2^1 + 0.2^0 + 1.2^{-1} + 1.2^{-2} = 4 + 2 + 0,5 + 0,25 = 6,75$

N = 111,101 est un nombre négatif dont l'opposé s'écrit :
-N = 000,010 + 1 (en réalité on fait " +0,001") autrement dit –N = 0,375 donc
$N = (111,101)_2 = (-0,375)_{10}$

A cause de la position fixe de la virgule, cette représentation est bien adaptée à l'addition mais pas à la multiplication (qui provoque un décalage de la virgule) d'où l'utilisation d'une autre représentation.

Expliquons la représentation de nombre en virgule fixe par un exemple.
Soit $(25,75)_{10} = (11001,110)_2$



La position de la virgule est **fixée** arbitrairement à la 4ème case vers la gauche. La position de la virgule n'est pas visualisée.
La case la plus à droite représente le poids 2^0 : ce qui est évidemment faux.
Cette représentation suppose la multiplication implicite de ce nombre par 2^{-3}. Le terme $^{-3}$ est représentatif du positionnement fixe de la virgule. Il devra impérativement être mémorisé.

b) Représentation en virgule flottante

Ici on écrit les nombres sous la forme exponentielle telle que $\mathbf{N = M.B^E}$ où B est la base, M la **mantisse** et E l'**exposant** (ou la **caractéristique**).
Si E > 0 on peut représenter des nombres très grand et si E < 0 des nombres très petits et la précision de la représentation dépend du nombre de bits de la mantisse.
La virgule est dite flottante pour la raison suivante : en notation pondérée courante, sa position dépend de la valeur de l'exposant.

Exemple : 1001,11 ne suffit pas à définir totalement cette donnée car il n'y a aucune indication sur la valeur du poids binaire affecté aux différents bits.
$N = 1001,11.2^0 = 100,111.2^1 = 10,0111.2^2 = 1,00111.2^3 = 0,100111.2^4$.

VI. ARITHMETIQUE BINAIRE

Les diverses opérations arithmétiques qui interviennent dans les ordinateurs et les calculatrices portent sur des nombres exprimés en notation binaire. En tant que telle, l'arithmétique numérique

peut être un sujet très complexe, particulièrement si on veut comprendre toutes les méthodes de calcul et la théorie sur laquelle elle s'appuie.
En ce qui nous concerne ici, nous allons concentrer nos efforts sur les principes de base qui nous permettent de comprendre comment les machines numériques (c'est-à-dire les ordinateurs) réalisent les opérations arithmétiques de base. Nous verrons comment effectuer manuellement les opérations arithmétiques en binaire, par la suite nous étudierons les circuits logiques réels qui matérialisent quelques unes de ces opérations dans un système numérique.

VI.1. Addition Binaire

L'addition de deux nombres binaires est parfaitement analogue à l'addition de deux nombres décimaux. En fait, l'addition binaire est plus simple puisqu'il y a moins de cas à apprendre.
Cependant, il n'y a que quatre cas, qui peuvent survenir lorsqu'on additionne deux chiffres binaires et cela quel que soit le rang. Ces quatre cas sont :
0 + 0 = 0
1 + 0 = 1
1 + 1 = 10 = 0 + report de 1 sur le rang de gauche
1 + 1 + 1 = 11 = 1 + report de 1 sur le rang de gauche.

Le dernier cas ne se produit que lorsque, pour un certain rang, on additionne deux 1 plus un report de 1 provenant du rang de droite.
Il n'est pas nécessaire d'étudier des additions ayant plus de deux nombres binaires, parce que dans tous les systèmes numériques les circuits qui additionnent ne traitent pas plus de deux nombres à la fois. Lorsque nous avons plus de deux nombres à additionner, on trouve la somme des deux premiers puis on additionne cette somme au troisième nombre, et ainsi de suite. Ce n'est pas véritablement un inconvénient, puisque les machines numériques modernes peuvent généralement réaliser une opération d'addition en quelques nanosecondes.
L'addition est l'opération arithmétique la plus importante dans les systèmes numériques. Les opérations de soustraction, de multiplication et de division effectuées par les ordinateurs ne sont essentiellement que des variantes de l'opération d'addition.

VI.2. Addition en complément à 2

La notation en complément à 2 et la notation en complément à 1 sont très semblables. Toutefois, la notation en complément à 2 jouit généralement de certains avantages quand vient le temps de construire des circuits.
Nous allons maintenant étudier comment les machines numériques additionnent et soustraient quand les nombres négatifs sont écrits dans la notation en complément à 2. Dans tous les cas étudiés, il est important que vous remarquiez que le bit de signe de chaque nombre est traité sur le même pied que les bits de la partie grandeur.

a) Cas 1: deux nombres positifs

L'addition de deux nombres positifs est immédiate. Soit l'addition de +9 et + 4 :

b) Cas 2: nombre positif et nombre négatif plus petit

Soit l'addition de + 9 et de -4. Rappelez-vous que -4 est exprimé dans la notation en complément à 2.

c) Cas 3: nombre positif et nombre négatif plus grand

Soit l'addition de -9 et de + 4 :

d) Cas 4: deux nombres négatifs

e) Cas 5: nombres égaux et opposés

VI.3. Soustraction et autres opérations: complément à 2

Une opération de soustraction qui porte sur des nombres exprimés dans la notation en complément à 2 est en réalité une opération d'addition qui diffère peu des cas examinés précédemment. Quand on soustrait un nombre binaire (le diminuteur) d'un autre nombre (le diminuande), la marche à suivre est comme sui t:

a. Prendre le complément à 2 du diminuteur, y compris son bit de signe. Si ce dernier est un nombre positif, il deviendra un nombre négatif dans la notation en complément à 2. Si le diminuteur est un nombre négatif, la complémentation à 2 en fera un nombre positif écrit en grandeur exacte. Autrement dit, nous changeons le signe du diminuteur.

b. Après avoir complémenté à 2 le diminuteur, on l'additionne au diminuande. Le diminuande conserve sa forme initiale. Le résultat de cette addition représente la différence recherchée. Le bit de signe de la différence indique si là réponse est positive ou négative et si on est en notation binaire exacte ou en notation en complément à 2.

c. Rappelez-vous que les deux nombres doivent avoir le même nombre de bits.

Examinons la soustraction suivante : + 9 - (+ 7).

a) Dépassement (overflow)

Dans chacun des exemples d'addition et de soustraction que l'on vient d'étudier, les nombres que l'on a additionnés étaient constitués à la fois d'un bit de signe et de 4 bits de grandeur. Les réponses aussi comportaient un bit de signe et 4 bits de grandeur. Tout report fait sur le bit de sixième rang était rejeté. Dans tous les cas étudiés, la grandeur de la réponse ne dépassait jamais la capacité des 4 bits.

Voyons l'addition de + 9 à + 8.

Le bit de signe de la réponse est celui d'un nombre négatif, ce qui est manifestement une erreur. La réponse devrait être +17. Étant donné que la grandeur est 17, il faut plus de 4 bits pour l'exprimer, et il y a donc un dépassement (over flow) sur le rang du bit de signe. Une condition de dépassement donne toujours lieu à un résultat inexact, et on la détecte en examinant le bit de signe du résultat et en le comparant aux bits de signe des nombres additionnés. En additionnant deux nombres de signes différents, il ne peu pas y avoir de dépassement, par contre lorsqu'on additionne deux nombres de même signe, on a un dépassement si le signe du résultat est différent du signe des deux nombres additionnés. Dans un ordinateur, il existe un circuit spécialement conçu pour détecter les conditions de débordement et pour indiquer que la réponse est fausse.

b) Multiplication en complément à 2

Dans les machines qui utilisent la notation en complément à 2, la multiplication est effectuée de la même façon que quand le multiplicande et le multiplicateur sont exprimés en notation binaire exacte. Si les deux nombres à multiplier sont positifs, ils sont déjà dans cette notation et ils sont multipliés tels quels. Le produit résultant est évidemment positif et son bit de signe est 0. Quand les deux nombres sont négatifs, ils sont donc écrits dans la notation en complément à 2. Chacun de ces nombres est complémenté à 2 pour obtenir un nombre positif et ce sont les résultats de ces complémentations qu'on multiplie. Le produit est un nombre positif dont le bit de signe est 0.
Quand un des nombres est positif et que l'autre est négatif, le nombre négatif est d'abord complémenté à 2 pour obtenir une grandeur positive. Le produit est exprimé selon la notation en grandeur exacte. Cependant, le produit doit être négatif car les nombres à multiplier sont de signes opposés. Par conséquent, on complémente à 2 le produit et on ajoute le bit de signe 1.

c) Division binaire et en complément à 2

La division d'un nombre binaire (le dividende) par un autre (le diviseur), comme on l'a déjà vu, est identique à la division de deux nombres décimaux. En réalité, la division en binaire est plus simple puisque pour déterminer combien de fois le diviseur entre dans le dividende, il n'y a que 2 possibilités 0 ou 1.
Dans la plupart des ordinateurs modernes, les soustractions qui ont lieu durant une opération de division sont généralement des soustractions avec complément à 2, c'est-à-dire on complémente à 2 le soustracteur puis on effectue l'addition.

La division de nombres signés s'effectue de la même façon que la multiplication. Les nombres négatifs sont complémentés pour obtenir des nombres positifs puis la division est effectuée. Si le dividende et le diviseur sont de signes opposés, le quotient est complémenté à 2 pour obtenir un nombre négatif, puis on lui ajoute un bit de signe de 1. Si le dividende et le diviseur ont le même signe, le quotient est laissé sous sa forme positive et on lui ajoute un bit de signe de 0.

EXERCICE N° 1

Représentez le nombre 8825 dans les bases 2, 3, 8, 9 et 16.
(Utilisez la technique des divisions successives pour les bases 2, 3 et 16.)

EXERCICE N° 2

1. Convertir $(110101101110)_2$ en base 16.
2. Convertir $(BAC0D)_{16}$ en base 2.
3. Transcoder les nombres suivants en base 2 : $(A9C)_{16}$.
4. Transcoder les nombres suivants en base 16 : $(1101101011)_2$.
5. Décoder les nombres suivants : $(1111001)_2$ et $(ABE1)_{16}$

EXERCICE N° 3
Réaliser les opérations suivantes :
11011 + 1101 ; 111010111 - 10111101 ;
1111 * 111 ; 10100/100.

Effectuer en hexadécimal l'addition de $(49B)_{16}$ et $(7AEC)_{16}$.
Effectuer en octal le produit de $(45)_8$ et $(76)_8$.

EXERCICE N° 4

Effectuez les conversions suivantes :
a) $B4F,D5_{(16)}$ ➔ Base 10
b) $324,21_{(5)}$ ➔ Base 10
c) $125_{(7)}$ ➔ Base 2
d) $73_{(8)}$ ➔ Base 16
e) $101101_{(2)}$ ➔ Base 8

EXERCICE N°5

Effectuer les additions de ces nombres représentés en complément à 2 :

1. 010101011 + 100110101
2. 10100110 + 11011011
3. 011001011 + 000111101
4. 100010010 + 101001011.

EXERCICE N°6

Un magicien sorcier qui s'était transformé en poule lors d'une de ces expériences, savait compter suivant un système de numération en base 5. Les cinq symboles qu'elle employait pour cela étaient C, T, O, E et D.
Quelle valeur numérique précise donnait-elle à chacune de ces cinq lettres, sachant que pour nommer l'entier 41346460, elle faisait COT COT CODET.

EXERCICE N°7

1- Déterminer la base A de la numération positionnelle dans laquelle $(82)_A = (28)_A$.
2- Dans quelles bases a-t-on :
   a- $32 \times 14 = 438$?
   b- $27 \times 35 = 708$?
3- Trouver l'écriture en base 16 du nombre $(4^3-1)(4^3+1)$ sans passer par la base décimale.

Opérations arithmétiques dans une base B

Pour l'addition et la soustraction, il faut prévoir une case des retenues comme on le verra dans les exemples ci-dessous. Tous les nombres de l'exemple sont en binaire ($B=2$).

1- Addition

⇒ la case retenue est au dessus de l'opération

```
 1 1 1 1 0   R
   1 1 0 1 1 0
 +     1 1 1 1
 -------------
 1 0 0 0 1 0 1
```

$(110110)_2 + (1111)_2 = (1000101)_2$

2- Soustraction

⇒ la case retenue est sous l'opération

```
     1 1 0 1 1 0
   -       1 1 1 1
   -------------
     1 1 1 1      R
   -------------
   1 0 0 1 1 1 1
```

$(110110)_2 - (1111)_2 = (100111)_2$

Pour ces 2 opérations il faut bien aligner les symboles (ou chiffres) comme en décimal

Toutes les opérations se font directement dans la base ⇒ Il est interdit de convertir d'abord en décimal

Les divisions et multiplications se font comme en décimal

```
     1 0 1
   x   1 1
   -------
     0 0 0    R
   -------
     1 0 1
   1 0 1
   -------
   1 1 1 1
```

```
 1 1 1 1 | 1 1
 1 1     | ---
   0 0   | 1 0 1
     1 1
```

$1111 / 11 = 101$

CHAPITRE 2 : CODAGE DES NOMBRES

Objectifs : Connaître les différents systèmes de codage des informations numériques

I. GENERALITES

Un code constitue une correspondance entre les symboles et les objets à désigner. C'est un langage composé de différents symboles ou mots.

Les codes sont des conventions d'organisation de caractères qui deviennent un langage (ou porteur d'information).

Le codage est une opération permettant de transmettre une information de la meilleure façon possible, à savoir :

- La plus simple,
- La plus rapide,
- La plus sécurisée (confidentialité).

En d'autres termes :

Le codage est l'opération qui transforme une information (écriture décimal, position angulaire, vitesse...) en écriture binaire dans un code de notre choix.

Un code est un langage composé de différents symboles ou mots.

Un mot est un ensemble de caractères numériques ou alphanumériques.

Quelques définitions

➲ Un codeur fait correspondre à l'activation d'une entrée particulière, une combinaison de bits en sortie (le circuit a donc une entrée et N sorties). Ainsi le codage 2n combinaisons en entrée nécessite n bits en sortie.

➲ Un décodeur réalise la fonction inverse, c'est-à-dire qu'il active une sortie particulière lorsqu'on lui présente une combinaison donnée de bits en entrée (le circuit a donc N entrées et une sortie). C'est le cas, par exemple, du clavier de porte automatique.

➲ Un transcodeur est un circuit qui permet de passer d'un code à un autre (N entrées et N sorties).

Les codes du chapitre 1 sont pondérés c'est à dire dans une base de travail, la valeur d'un rang donné est un multiple par la base de celle du rang inférieur.

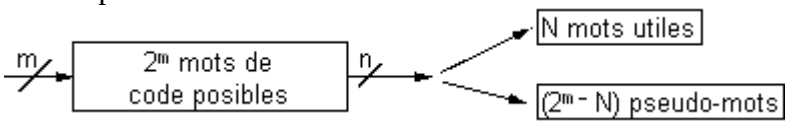
Un code est dit « pondéré » quand il existe des nombres qui indiquent le poids en chiffres (binaires dans le cas de la base 2) ; en multipliant ces nombres par les chiffres (binaires) correspondants, on obtient l'équivalence décimale.

Tous les autres codes dans lesquels on ne peut pas repérer le poids des chiffres (binaires) sont appelés « non pondérés ». Autrement dit que la position d'écriture ne correspond pas à un poids des autres. Ces codes ne permettent d'effectuer des opérations arithmétiques.

Ils sont élaborés sur une base à développement mathématique complexe ou plus simplement sont caractérisés par des tables faites spécialement.

Différents codes ont été imaginés, ayant diverses propriétés logiques et arithmétiques. Le choix de l'un ou de l'autre type de code dépend exclusivement des applications auxquelles il est destiné.

Chaque code présente deux attributs caractéristiques : la longueur **m** et le nombre de mot de code **n** possible.



Pour le code BCD (voire paragraphe suivant), par exemple, la longueur est m=4 et le nombre de mots de code (mots utiles) N=10.

Il faut prendre en considération la différence entre un code complet et un code incomplet. Le code complet contient tous les mots de code binaire ou combinaisons possibles. Par contre, le code incomplet n'utilise qu'une partie des combinaisons possibles.

II. CODES PONDERES

II.1. Code naturel

Le code binaire naturel et ses dérivées (octal et hexadécimal entre autres) répondent aux règles classiques de l'arithmétique des nombres positifs, on peut donc calculer.

Les **caractères** sont des bits, **les mots** sont formés par une association ou combinaison de bits et avec n bits nous pouvons former au maximum 2^n mots.

Dans les systèmes numériques, l'entrée des données ne s'effectue pas en pur code binaire. Ce sont plutôt des nombres décimaux codés en binaire (code BCD) qui sont utilisés. BCD (Décimal Codé en Binaire) signifie que les nombres décimaux 0 à 9 sont représentés par un nombre binaire à plusieurs chiffres.

A cet effet, il est possible de choisir n'importe qu'elle correspondance entre les nombres décimaux et les nombres binaires.

II.2. <u>Code "Décimal Codé Binaire" (DCB)</u>

Chacun des 10 chiffres 0 à 9 peut être représenté dans le code binaire pur par un mot de quatre (4) bits (tétrades).

Dans ce codage DCB (ou BCD en anglais : Binary Coded Decimal) chaque digit décimal est écrit en binaire puis tous sont juxtaposés.

Exemple : 0110 0111 0011 est le code de $(673)_{10}$.

Cette représentation est commode pour traiter les nombres dans le mode de représentation le plus adapté à l'opérateur humain (loi de l'affichage par exemple). Ce code est aussi appelé ***code 8421*** ou ***code 1248***.

Le code DCB est un code incomplet car n'utilisant qu'une partie des combinaisons possibles (10 mots de codes binaire sur 16).

Nombre décimal	Code binaire m = 4				Code DCB N = 10			
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1
4	0	1	0	0	0	1	0	0
5	0	1	0	1	0	1	0	1
6	0	1	1	0	0	1	1	0
7	0	1	1	1	0	1	1	1
8	1	0	0	0	1	0	0	0
9	1	0	0	1	1	0	0	1
10	1	0	1	0	Pseudo-tétrades			
11	1	0	1	1				
12	1	1	0	0				
13	1	1	0	1				
14	1	1	1	0				
15	1	1	1	1				

Voici comment on peut effectuer l'addition en BCD. Si l'on additionne en arithmétique binaire les codes BCD correspondant à des chiffres dont la somme ne dépasse pas 9, on aura évidemment un résultat donnant la représentation correcte de la somme DCB. Par exemple, 45 + 31 donnent bien 76 :

45	0100 0101
+31	+0011 0001
76	0111 0110

Cependant, dès que la somme dépasse 9 dans un motif de 4 bits, il faut apporter une correction. Par exemple, si on additionne 8 + 6, on obtient 1110, qui excède l'intervalle de définition d'un digit BCD. On voit cependant que si on ajoute 6 à ce résultat, on obtient bien la représentation BCD de 14:

```
  8          1000
 +6         +0110
----   =    ------
 14          1110  Résultat > 9,
            +0110  on ajoute 6
            ------
           1 0100  = 0001 0100 = 14BCD
```

On effectue donc l'addition en motifs de 4 bits. Si le résultat dépasse 9 pour l'un ou plusieurs de ces motifs, on leur ajoute 6 pour forcer une retenue et on obtient le résultat escompté en BCD :

```
 76   0111 0110      76   0111 0110      26   0010 0110
+45   0100 0101     +43   0100 0011     +45   0100 0101
---  ----------     ---  ----------     ---  ----------
121   1011 1011     119   1011 1001      71   0110 1011
     +0110 0110          +0110                  +0110
     ----------          ----------             ----------
     1 0010 0001         1 0001 1001            0111 0001
```

II.3. <u>Code 2421 – Code AIKEN</u>

Parmi les variétés de codes possibles, le code 2421 est aussi utilisé. Dans ce code, les poids affectés aux variables binaires ne sont plus 8, 4, 2, 1 mais 2, 4, 2, 1.

Dans ce code il existe deux manières de représenter certains chiffres ainsi :

- "7" peut s'écrire *0111* ou <u>1101</u>
- "5" peut s'écrire *0101* ou <u>1011</u>

Donc il y a deux représentations pour ce type de code.

La première représentation (en italique) est appelée code 2421, et la seconde (soulignée) est le *code 2421 AIKEN* ou simplement ***code AIKEN***.

❖ Ces deux codes sont pondérés.

❖ On remarque que le chiffre « 9 » est représenté par quatre (4) bits à 1 ==> donc on ne peut pas représenter une valeur supérieure à 9 avec 4 bits en code 2421 contrairement au 8421 qui va jusqu'à 16.

❖ On remarque aussi que pour les chiffres décimaux 0, 1, 2, 3 et 4 le code AIKEN correspond avec le code BCD 8421, tandis que pour les chiffres décimaux 5, 6, 7, 8 et 9 il concorde avec les nombres 11, 12, 13, 14 et 15 du code binaire pur.

❖ Le code AIKEN a la propriété d'être auto-complémentaire c'est-à-dire qu'on obtient le complément à 9 du nombre codifié en remplaçant les "0" par des "1" et les "1" par des "0". Ainsi le complément à 9 de 6 (1100) est 3 (0011).

❖ On remarque aussi que la dernière variable pondérée 2 dans le code AIKEN est égale à <u>0</u> si le nombre est inférieur à 5 et est égale à <u>1</u> si le nombre est supérieur ou égal à 5.

III. CODES NON PONDERES

III.1. Code Gray (ou binaire réfléchi)

Dans ce code, un seul bit (ou digit) change entre deux valeurs adjacentes, c'est-à-dire entre deux nombres décimaux successifs N et N+1. Il est employé dès que l'on doit représenter une évolution réelle des variables où une seule change à un instant. De plus, on opère de telle façon que le bit de transformation soit d'un poids faible. Ainsi si une erreur survient lors d'une transformation d'un nombre à un autre, elle est ainsi minimisée.

Commençons par un exemple simple et établissons le code gray pour les 4 premiers chiffres décimaux 0 à 3. Deux bits suffisent et les combinaisons en binaire BCD sont les suivantes :

```
0 | 0 0
1 | 0 1
2 | 1 0 : ici nous avons inversé la valeur de 2 bits à la fois
3 | 1 1
```

Nous remarquons que pour aller du nombre $(1)_{10}$ à $(2)_{10}$ nous changeons les deux bits à la fois pour passer de $(01)_2$ à $(10)_2$.

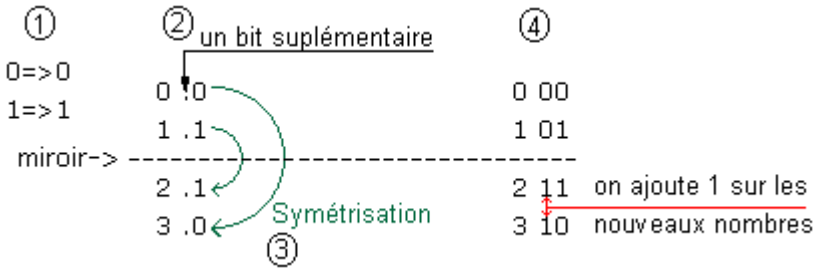
En code Gray, pour passer d'une ligne à la suivante, on inverse **un seul bit** de telle manière qu'il soit le bit **le plus à droite possible** conduisant à **un nouveau nombre**. Ce qui donne les combinaisons suivantes:

```
0 | 0 0 : on commence à zéro,
1 | 0 1 :
2 | 1 1 :  } on inverse un seul bit le plus à droite possible conduisant
3 | 1 0 :  } à un nouveau nombre.
```

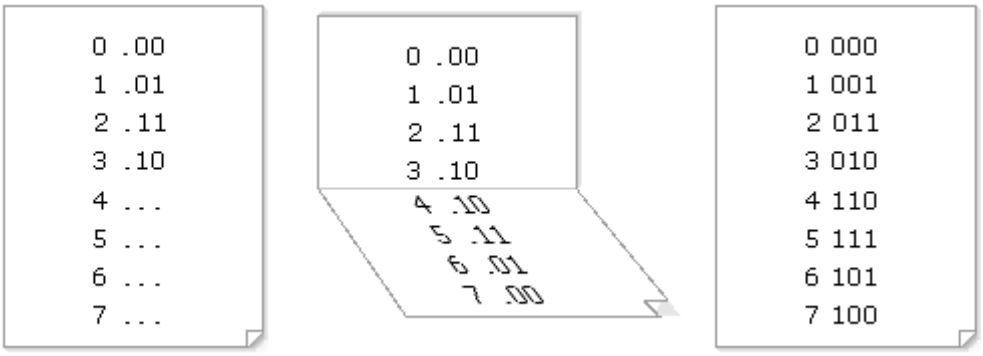
Construction par la méthode du code binaire réfléchi ou code REFLEX

Le nom **code binaire réfléchi** vient d'une autre méthode de construction. Elle est plus pratique ou plus visuelle quand au choix du bit à inverser lors du passage d'un nombre au suivant.

On établit un code de départ: zéro est codé 0 et un est codé 1 (1).Puis, à chaque fois qu'on a besoin d'un bit supplémentaire (2), on symétrise les nombres déjà obtenus (comme une réflexion dans un miroir)(3) et on rajoute un 1 au début des nouveaux nombres (4) et un zéro sur les anciens.



Ci-dessous nous recommençons l'expérience pour les nombres $(0)_{10}$ à $(7)_{10}$.



Règle de formation du code Gray à partir du binaire pur

Soit un nombre N en binaire pur, pour obtenir son équivalent n en binaire réfléchi, il suffit d'effectuer l'opération suivante :

$$n = \frac{N \oplus 2N}{2}$$

Exemple :
Soit N = 0111, nous avons 2N = 1110 (pour multiplier par 2 on effectue un décalage de la droite vers la gauche). On effectue maintenant l'opération OU Exclusif de N et 2N :

$$\begin{array}{r} 0111 \\ \oplus\ 1110 \\ \hline 1001 \end{array}$$

Puis nous divisons par 2 le résultat soit 1001 / 2 = 0100 (pour diviser par 2 on effectue un décalage de la gauche vers la droite). Nous avons alors : pour N = 0111 en binaire pur correspond n = 0100 en code Gray.

Nombre décimal	Code binaire pur				Code Gray			
	B4	B3	B2	B1	G4	G3	G2	G1
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

III.2. Code complément à 9

Ce code concerne les chiffres de 0 à 9. A partir du binaire naturel, un nombre N (<9) est codé par le binaire de (9-N)

$(0)_{10} = (0000)_2 = (1001)_{\text{complément à 9}}$ et $(5)_{10} = (0101)_2 = (0100)_{\text{complément à 9}}$

Nombre décimal	Code 8421 (8) D	(4) C	(2) B	(1) A	Code complément à neuf D'	C'	B'	A'
0	0	0	0	0	1	0	0	1
1	0	0	0	1	1	0	0	0
2	0	0	1	0	0	1	1	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	0	0
6	0	1	1	0	0	0	1	1
7	0	1	1	1	0	0	1	0
8	1	0	0	0	0	0	0	1
9	1	0	0	1	0	0	0	0

III.3. Code excédent 3 (ou code plus 3)

Le code plus 3 (ou code excess 3) appelé aussi code STIBIZ du nom de son inventeur est un code non pondéré issu du code 8421 auquel on ajoute systématiquement 3 à chaque chiffre.

Le code plus 3 est souvent utilisé sur des unités arithmétiques qui calculent en système numérique décimal plutôt qu'en système binaire. Ce code permet d'effectuer les opérations arithmétiques d'addition et de soustraction avec un minimum de fonctions logiques.

Le code plus 3 utilise les combinaisons de quatre chiffres du binaire pur compris entre les nombres décimaux 3 et 12. Chaque nombre s'obtient en ajoutant 3 à chaque chiffre du nombre décimal et en le codant en BCD 8421.

Dans ce système n'apparaissent pas les combinaisons 0000 et 1111. De plus ce code est auto complémentaire.

Exemple :

Le complément de 7 à 9 est 2

7 = 1010

2 = 0101

Nombre décimal	Code 8421				Code plus 3			
	(8) D	(4) C	(2) B	(1) A	D'	C'	B'	A'
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

III.4. Codes détecteurs et correcteurs d'erreurs (codes redondants)

Il peut arriver que la transmission d'une information se fasse avec une ou plusieurs erreurs de bits (changement d'un 0 en 1 ou 1 en 0). Et la "fausse" information est aussi vraisemblable que la "vraie" donc dans ce cas l'erreur n'est pas décelable.

Ces codes permettent alors de détecter une erreur de bits et parfois même une correction de l'erreur.

Comme exemple, nous pouvons citer les codes p parmi n et le code de Hamming.

Pour lever cette contrainte, on peut imposer pour coder une information, l'utilisation d'un nombre limité de combinaison de 0 et 1. On a ainsi une chance de déceler immédiatement la combinaison erronée.

Le code le plus simple est le code détecteur d'erreur qu'on nomme *contrôle de parité*. Dans ce code pour représenter les 10 chiffres décimaux (0 à 9), on retient un mot de 5 bits et on donne au 5ème bit la valeur 0 ou 1 de telle sorte que le nombre de 1 parmi les 5 variables soit toujours pairs (c'est le code 2 parmi 5)

III.5. Code Hexadécimal

Ce code très utilisé dans les microprocesseurs.

Exemple : 3DE = 001111011110.

III.6. Code non numérique

Ici les mots peuvent avoir une signification non numérique. Afin de pouvoir transmettre ou stocker tous les types de caractères alphanumériques ou autres, des codes conventionnels ont été établis. Chaque caractère est associé à son équivalent en code numérique. Il existe de nombreux codes et nous pouvons citer pour mémoire le code ASCII, l'EBCDIC, L'UNICODE, l'UTF8... Ce type de codes le plus connu et le plus utilisé en informatique est le code ASCII (American Standard Code for Information Interchange).

EXERCICE N° 1

1- Coder les nombres suivants en BCD : $(62)_{10}$ et $(59)_{10}$

2- Les deux nombres ci-dessous sont des nombres DCB, déterminer leur équivalent décimal.

1001011101010010 et 0110000100

EXERCICE N° 2

Convertir les nombres binaires suivant en nombres en Code Gray.
A) $(100110)_2$ B) $(111001)_2$ C) $(100111)_2$

Convertir les nombres en Code Gray suivant en nombres binaires.
D) $(101101)_{Gray}$ E) $(1100111)_{Gray}$ F) $(1001101)_{Gray}$

EXERCICE N° 3

Effectuez les opérations :

001111100010_{BCD}+ 100100010101_{BCD} ; 01110111_{BCD} + 01101001_{BCD} ;

CHAPITRE 3 : OPERATEURS LOGIQUES ELEMENTAIRES

Objectifs :

- Connaître les différents opérateurs logiques.
- Savoir les identifier et les utiliser dans des circuits logiques simples.

I. DEFINITIONS

I.1. Variable binaire

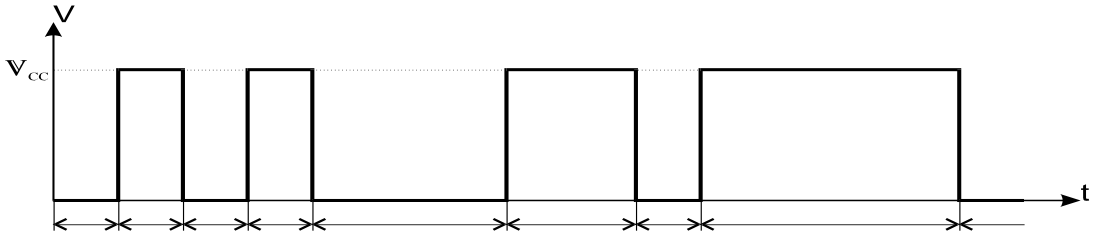
En électronique numérique, nous sommes confrontés, contrairement à l'électronique analogique, à uniquement deux valeurs possibles pour les signaux.

En supposant nos montages alimentés avec une différence de potentiels égale à V_{CC}, les deux valeurs possibles sont : le 0 volt (la masse) et le $+V_{CC}$, car nous travaillons dans une logique dite *positive.*

Ces deux valeurs possibles sont associées à des états logiques appelés :

* État bas ou « 0 » pour le 0V (en anglais : L pour Low) ;
* État haut ou « 1 » pour $+V_{CC}$ (en anglais : H pour High).

On peut ainsi associer des valeurs logiques aux différentes valeurs du signal suivant. Complétez celui-ci en indiquant sur chaque flèche le niveau logique correspondant :



Une variable binaire (ou logique) est une variable qui peut prendre, à tout instant, une valeur unique dans l'ensemble {0, 1}.

Exemple :

- état d'un interrupteur (fermé ou ouvert)

- état d'un bouton poussoir (haut ou bas)

- état d'une vanne (fermé ou ouvert)

- Un ensemble ordonné de n variables binaires est un *mot binaire* de n digits (ou bits).

Exemple : un mot binaire à deux variables logiques est noté (a, b) ou ab.

- Un ensemble binaire de n variables logiques peut prendre au plus 2^n valeurs distinctes.

Exemple : le mot binaire ab peut prendre 4 valeurs notées : 00, 01, 10, 11

- On appelle *état binaire*, la valeur d'un mot à un instant donné.

- On appelle variable tout terme indéterminé qui peut être remplacé par divers termes déterminés qui en sont les valeurs.

I.2. Fonction logique (ou équation logique)

Une fonction logique f est une fonction qui à un mot binaire $y = x_n \ldots x_1$, associe une variable binaire z.

C'est une combinaison de plusieurs variables logiques donnant l'état d'une variable dite de sortie associée. Cette combinaison est réalisée à l'aide d'opérateurs logiques.

Soit $x_i \left(i \in [1,n]\right)$ les variables d'entrées, l'équation ou fonction $A = f(x_i)$ définit l'état de la variable de sortie A.

Une fonction logique est combinatoire si et seulement si l'état de la sortie à chaque instant ne dépend que de l'état du mot d'entrée à cet instant. Dans ce cas la fonction est indépendante du temps et on a : $s(t) = f(e(t)) \ \forall t$.

Un système combinatoire est un système comprenant plusieurs fonctions combinatoires. A partir d'un mot binaire d'entrée, le système élabore plusieurs variables de sortie. Le système combinatoire ne doit donc pas présenter de réactions de la sortie sur l'entrée, de sorte à ce que l'état de la sortie ne dépende pas de l'histoire du système.

Remarque :

Pour une proposition dépendant de n variables binaires, il y a $N = 2^n$ combinaisons possibles.

Cette proposition peut être vraie ou fausse pour chacune des combinaisons ==> il y a 2^N résultats soit $(2)^{2^n}$ opérations différentes.

- Pour le cas n = 1 il y a 4 fonctions possibles (fonctions à une variable)

- Pour le cas n = 2 il y a 16 fonctions possibles (fonctions à deux variables), mais seul un petit nombre intervient dans la réalisation technologique associée à la structure d'algèbre de Boole (algèbre des variables et fonctions binaires).

I.3. Table de vérité

Une table de vérité représente l'état de la variable de sortie pour chacune des combinaisons des n variables d'entrée (2^n lignes). Autrement dit une table de vérité nous fait connaître la réaction d'un

circuit logique (sa valeur de sortie) aux diverses combinaisons de niveaux logiques appliqués aux entrées.

I.4. Théorie

Nous savons tous que les ordinateurs fonctionnent en utilisant exclusivement deux valeurs numériques : 0 ou 1...

Les raisonnements logiques peuvent se ramener à des choix "binaires", en fonction des réponses "oui" ou "non" à quelques questions judicieuses.

Est-il possible d'appliquer ces deux idées de base à une machine et lui apprendre ainsi à raisonner?

Exemple :

La proposition de base est : "SI tu as 500 FCFA ET SI tu as faim, ALORS, tu iras au restaurant universitaire"

On peut décomposer le problème de la façon suivante :
Question 1 : "As-tu de 500 FCFA ?"
Question 2 : "As-tu faim ?"

On vous demande de déduire (dans le tableau ci-dessous) des réponses à ces deux questions la réponse à la question 3 :

Question 3 : "Iras-tu manger au restaurant universitaire ?"

Cet ensemble de deux réponses permettant de trouver logiquement la troisième peut se symboliser par un tableau,

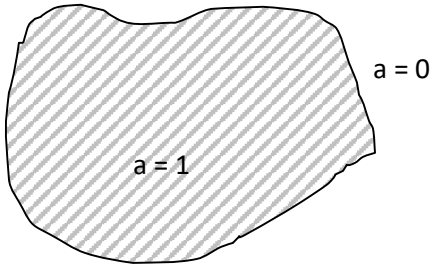
Question 1	Question 2	Question 3
oui	oui	...
oui	non	...
non	oui	...
non	non	...

a) Complétez ce tableau appelé "table de vérité".

En remplaçant les réponses OUI et NON par les variables logiques correspondantes, on obtient la table de vérité d'une fonction logique qu'on verra plus loin.

II. OPERATEURS LOGIQUES ELEMENTAIRES (OU DE BASE)

Ils servent à construire les fonctions logiques. Pour interpréter les opérations logiques, on peut utiliser la théorie des ensembles par le biais du diagramme de Venn (ou d'Euler-Venn). A chaque variable logique, on associe une courbe fermée dont l'intérieur représente l'ensemble des cas où la proposition est vraie (variable = 1) et l'extérieur l'ensemble des cas où elle est fausse (variable = 0).



Un **logigramme** d'une opération (ou opérateur) est le symbole (ou figure) qui schématise le circuit électronique qui réalise cette opération.

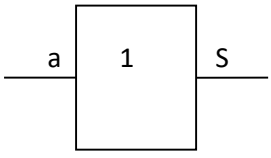
Un **chronogramme** est une représentation graphique qui permet de visualiser en fonction du temps toutes les combinaisons d'états logiques possibles des entrées avec l'état correspondant de la sortie

II.1. Opérateur OUI (ou identité)

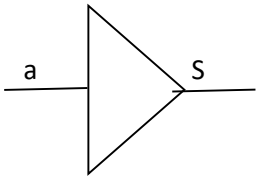
L'opération (ou opérateur) OUI est dite **unaire** (ne s'applique qu'à un seul opérande). Elle affecte à la variable de sortie l'état logique de la variable d'entrée.

Equation : a est la l'entrée, *S* la sortie : $S = a$.

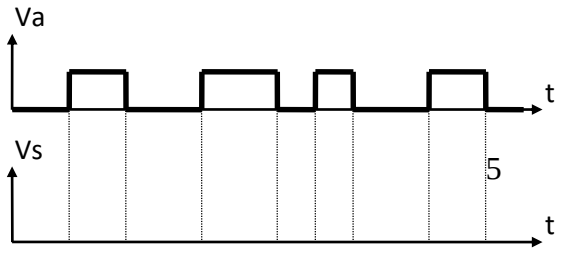
a	S
0	
1	



Norme IEC ou CEI



Norme IEEE ou MIL (USA)



Cette figure (à gauche)
représente le **chronogramme**

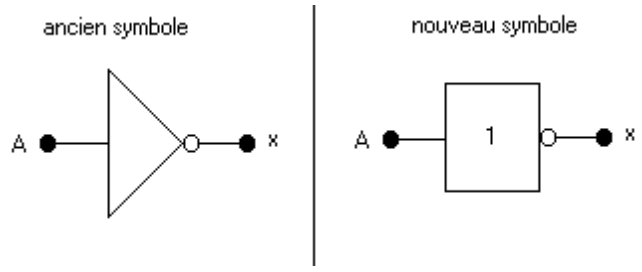
IEC = International Electrotechnical Commission (CEI en français)

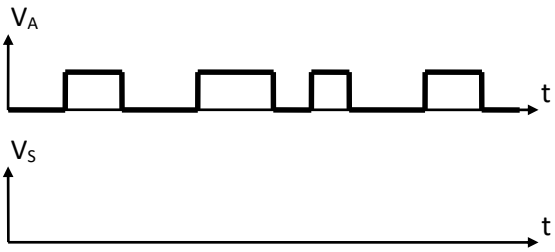
IEEE = Institute of Electrical and Electronic Engineers

II.2. Opérateur NON

L'opération (ou opérateur) NON, encore appelée *complément, inverse, négation ou Pas*, est la fonction unaire qui affecte à la variable de sortie l'état complémentaire de la variable d'entrée.

Equation : *A* est la l'entrée, S=*x* la sortie, $x = \bar{A}$ (prononcer « *A* barre »).

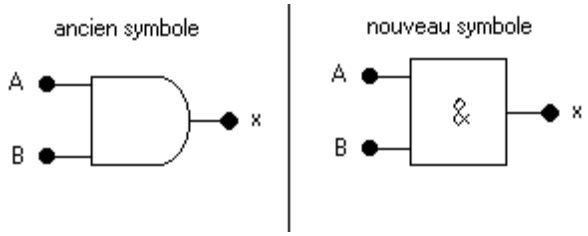


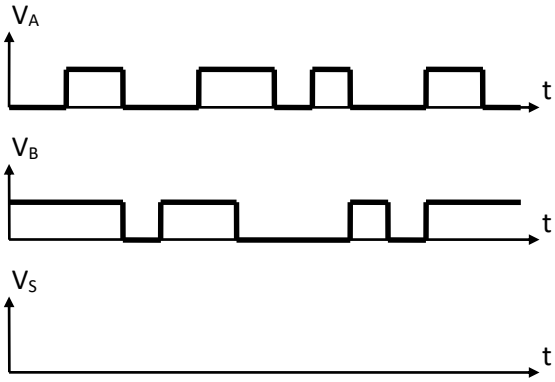


II.3. Opérateur ET

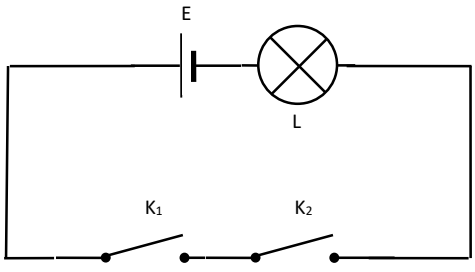
L'opération ET est le **produit logique**. Le signe est celui de la multiplication (un point), mais on lit « et ». C'est un opérateur **binaire** qui affecte à la variable de sortie l'état **1** si et seulement si les variables d'entrée sont à **1** simultanément.

Equation : *A* et *B* les entrées, *S* la sortie, $S = x = A.B = B.A$. On note aussi l'opération ET par un V retourné : $A.B = A \wedge B$ (penser à l'intersection d'ensembles).





Remarque : La fonction **ET** peut également être expliquer à partir du montage électrique suivant.



L est allumée si K_1 **ET** K_2 sont fermés.

On peut définir deux variables logiques **a** et **b** qui représentent les états des interrupteurs K_I et K_2

a = 0 si I_I est ouvert, b =0 si I_2 est ouvert.

a = 1 si I_I est fermé, b =1 si I_2 est fermé.

On peut définir une fonction logique **S** (sortie) qui représente l'état de la lampe L :

Si L est éteinte => S = 0

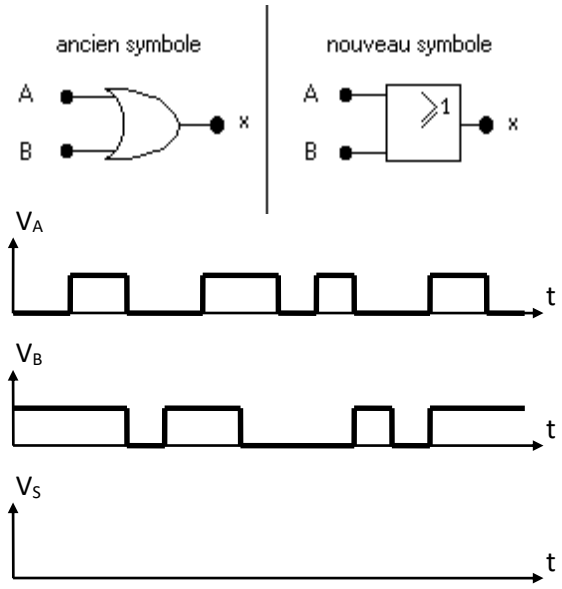
Si L est allumée =>S= 1

On peut alors définir une table de vérité qui permet de connaître la valeur de la fonction logique S en fonction des diverses combinaisons des valeurs des variables **a** et **b**. Les valeurs des variables sont placées dans l'ordre binaire naturel. La table de vérité traduira le fonctionnement du système.

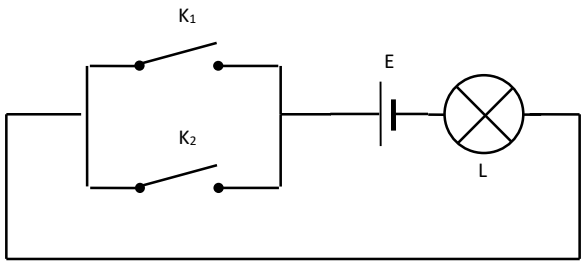
II.4. Opérateur OU

L'opération OU est la **somme logique**. Le signe est celui de l'addition (+), mais on lit « ou ». C'est un opérateur binaire qui affecte à la variable de sortie l'état **1** si et seulement si une variable d'entrée est à **1**. Cette définition induit directement le symbole ≥1.

Equation : *A* et *B* les entrées, S la sortie, S = *x* = *A* + *B* = *B* + *A*. On note aussi l'opération OU par un V : *A* + *B* = *A* ∨ *B* (penser à l'union d'ensembles).



La fonction **OU** peut également être expliquer à partir du montage électrique suivant.



Remarques :

Il est possible d'étendre la notion d'opération logique en utilisant des concepts plus « algébriques » :

- Pour le NON logique : $\bar{a} = 1 - a$,
- Pour le ET logique : $x \cdot y = \text{Min}(x, y)$,
- Pour le OU logique : $x + y = \text{Max}(x, y)$

Avec a, x et y des variables logiques donc appartenant à l'ensemble {0, 1}. Ces notations sont aisément vérifiables à l'aide de tables de vérité.

III. OPERATEURS LOGIQUES INDUITS

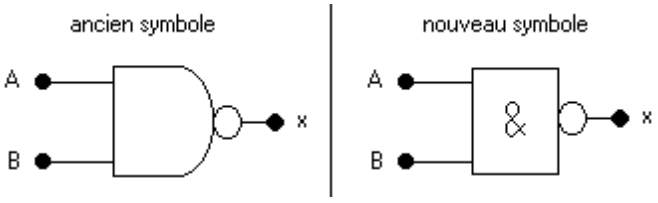
On peut montrer qu'il est possible de synthétiser tous les opérateurs de base ci-dessus avec un seul type d'opérateurs que on l'appelle ***opérateurs complets ou induits ou universels***. Il en existe deux : NAND et NOR.

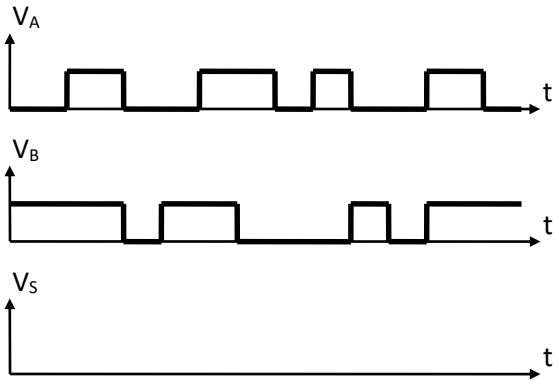
III.1. Opérateur NON-ET (ou NAND)

Cette fonction logique est le résultat de l'association d'un NON et d'un ET. C'est un opérateur binaire qui affecte à la variable de sortie l'état **0** si et seulement si les variables d'entrée sont à **1** simultanément.

Equation : *A* et *B* les entrées, S la sortie, S = $x = \overline{A \cdot B}$

On note aussi l'opération NAND par une flèche montante.

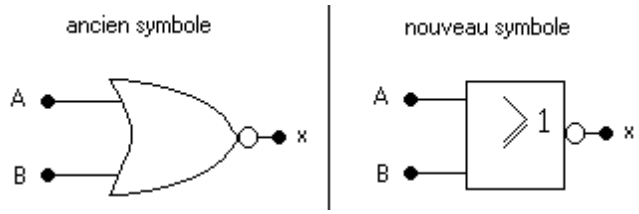


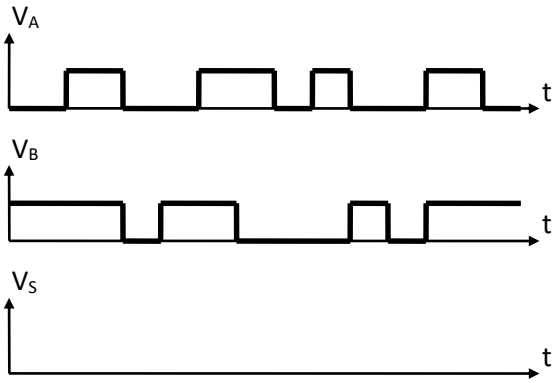


III.2. <u>Opérateur NON-OU (ou NOR ou "NI")</u>

Cette fonction logique est le résultat de l'association d'un NON et d'un OU. C'est un opérateur binaire qui affecte à la variable de sortie l'état **1** si et seulement si les variables d'entrée sont à **0** simultanément.

Equation : *A* et *B* les entrées, *x* la sortie, $x = \overline{A + B}$. On note aussi l'opération NOR par une flèche descendante.





III.3. Opérateur OU-EXCLUSIF (ou XOR)

Cet opérateur logique binaire ne prend la valeur **1** que si une seule des entrées est à **1**.

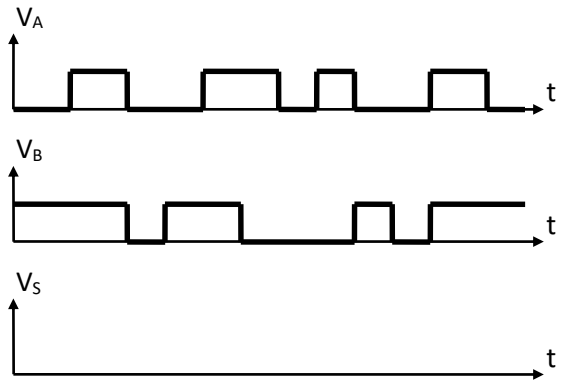
Cette opération est notée S = $x = A \oplus B = \overline{A}.B + A.\overline{B}$.

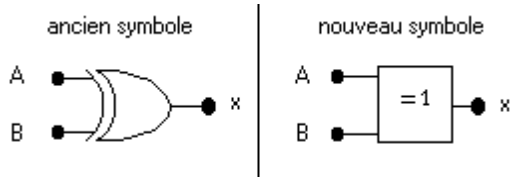
Généralisation

L'opérateur XOR se généralise à un ensemble de *n* variables d'entrée par la définition suivante :

La sortie vaut 1 si et seulement si le nombre d'entrées à 1 est impair.

Cet opérateur peut donc aisément faire fonction de contrôleur de parité (ou d'imparité).





III.4. Opérateur NON-OU-EXCLUSIF (ou XNOR)

Cette fonction logique est le résultat de l'association d'un NON et d'un OU-EXCLUSIF.

De manière générale, la fonction de sortie d'une porte NON-OU-EXCLUSIF indique l'égalité entre les deux signaux d'entrées. La sortie est active (niveau 1) si l'état logique des entrées est **identique**. Le fonctionnement de cette porte correspond à un comparateur un bit.

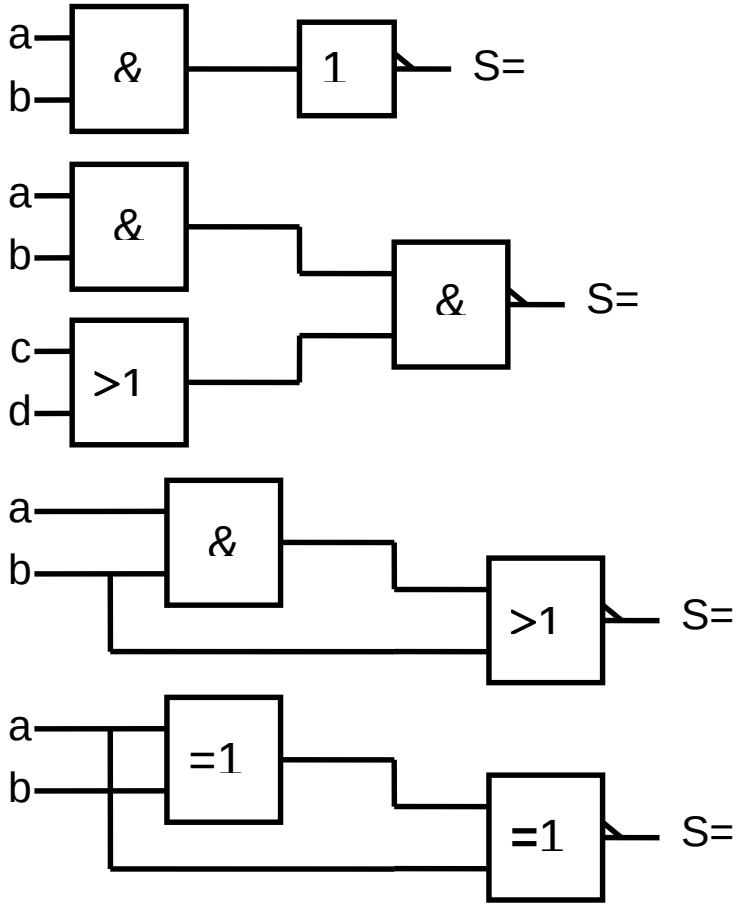
Cet opérateur a une propriété particulière. L'inversion de la sortie peut être reportée sur l'une ou l'autre des entrées, soit : $\boxed{\overline{A \oplus B} = \overline{A} \oplus B = A \oplus \overline{B}}$ à démontrer en TD

Equation, table de vérité, logigramme et chronogramme à faire.

III.5. Association d'opérateurs

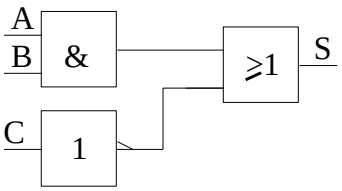
Il est possible d'associer des opérateurs en les connectant en « cascade ».

Un opérateur, dont l'une des entrées est reliée à la sortie d'un autre opérateur, se verra appliquée sur son entrée le résultat obtenu sur la sortie de l'opérateur précédent. On combinera chaque nouvelle valeur des entrées dans l'équation de celui-ci pour obtenir l'équation de sortie.



EXERCICE N° 1

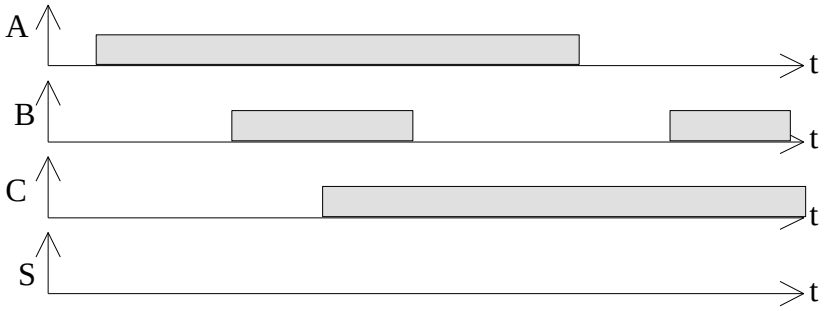
Soit la fonction définie par le schéma structurel ci-contre:



a) établir l'équation logique de S
b) tracer la table de vérité de la S

c) compléter le chronogramme de S

C	A	B	S



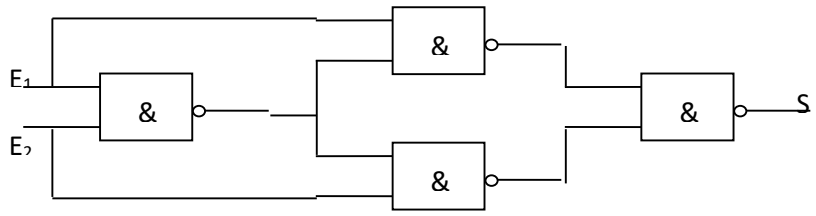
EXERCICE N° 2

Compléter la table de vérité suivante :

A	B	C	$\bar{A}$	$\bar{B}$	$\bar{C}$	$X=A.B.\bar{C}$	Y=	Z=X+Y	....
0	0	0	1	1	1		0		0
0	0	1	1	1	0		0		1
0	1	0	1	0	1		0		0
0	1	1	1	0	0		0		1
1	0	0	0	1	1		1		0
1	0	1	0	1	0		0		0
1	1	0	0	0	1		0		0
1	1	1	0	0	0		0		1

EXERCICE N° 3

Soit le logigramme ci-après :



Déterminer l'équation logique de la sortie et sa table de vérité.

→ Conversion de binaire au code Gray

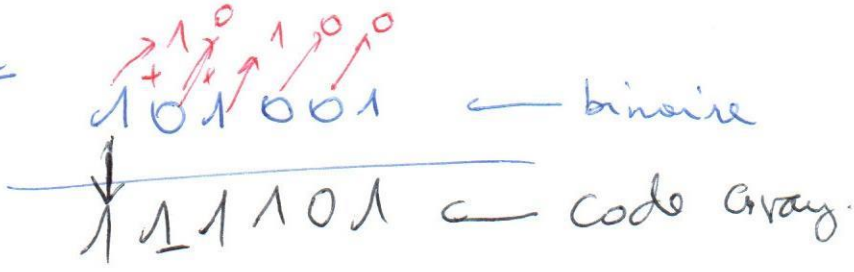
Cette conversion se fait de la façon sot.

1→ le bit du poids le plus fort reste le même

2→ De la gauche vers la dte, chaque bit est additionné à son voisin de dte, la somme fait partie du code Gray. Les retenues sont oubliées

3→ le résultat compote le m nbre de bits que le nbre binaire de départ.

Exple



→ Conversion du code Gray à binaire

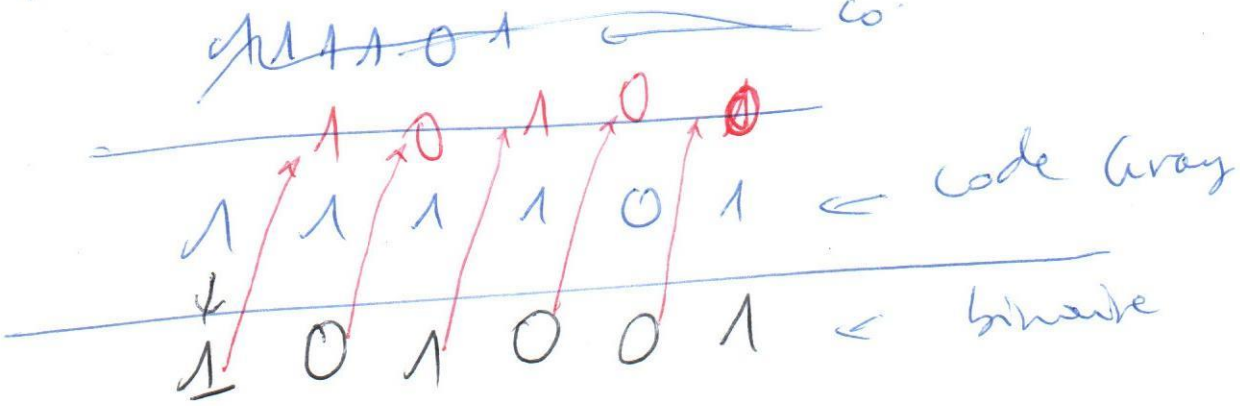
Cette conversion se fait de la façon ste;

1→ le bit du poids le plus fort reste le même

2→ De la gauche vers la dte, on additionne la somme binaire à son voisin de dte. Les retenues sont oubliées.

3 - le résultat compote le m nbre de bits que le nbre binaire de départ

Exple



I-2- Fonction logique

Remarque :

- Pour le cas $n=1$

Variable	Fonctions			
A	F_0	F_1	F_2	F_3
0	0	0	1	1
1	0	1	0	1
	toujours faux	identité	Négation	toujours vrai

- Pour le cas $n=2$

Variables		Fonctions															
A	B	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_A	F_B	F_C	F_D	F_E	F_F
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

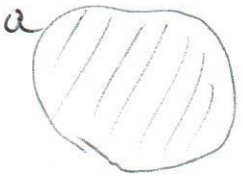
I-4

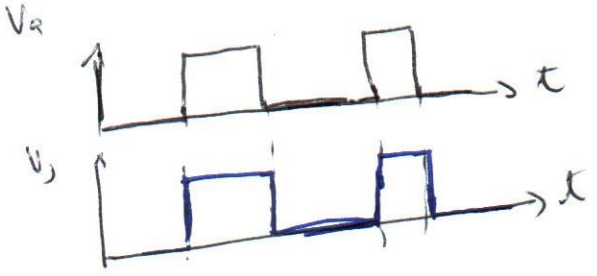
Exemple

Question 1	Question 2	Question 3
oui	oui	oui
oui	non	non
non	oui	non
non	non	non

II-1 Opérateur OUI

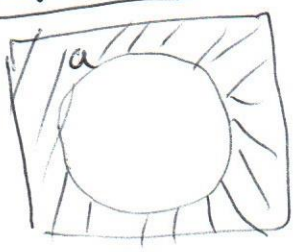
a	S
0	0
1	1

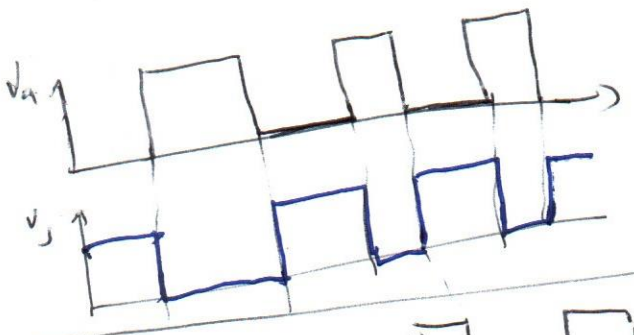




II-2 Opérateur NON

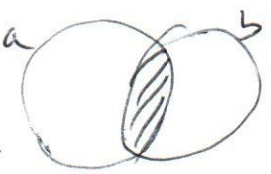
a	S
0	1
1	0

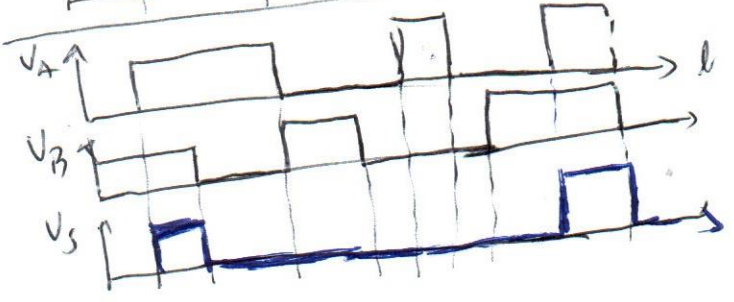




II-3 Opérateur ET

a	b	S
0	0	0
0	1	0
1	0	0
1	1	1





CHAPITRE 4 : FONCTIONS ET CIRCUITS LOGIQUES COMBINATOIRES COMPLEXES

Objectifs : Connaitre les méthodes de simplification des fonctions logiques

Savoir réaliser les fonctions logiques à l'aide de portes.

I. SIMPLIFICATION DES FONCTIONS LOGIQUES

Tous les opérateurs précédents permettent de combiner des variables pour en construire de nouvelles. La relation entre les grandeurs d'entrée et la sortie se présente souvent sous une forme algébrique très complexe. Pour limiter le nombre de portes nécessaires à la réalisation de ces fonctions, il est parfois nécessaire de simplifier l'équation liant les grandeurs entre elles. Nous allons nous intéresser aux différentes méthodes permettant ces simplifications.

Exemple : $\left.\begin{array}{l} c = a + b.d \\ d = e + f \end{array}\right\} \Rightarrow c = a + b.(e + f)$

I.1. Propriétés

· ***Commutativité***

$a + b = b + a$ (commutativité de l'opération OU)

$a.b = b.a$ (commutativité de l'opération ET)

· ***Associativité***

$a + (b + c) = (a + b) + c = a + b + c$ (associativité de l'opération OU)

$(ab)c = a(bc) = abc$ (associativité de l'opération ET)

· ***Distributivité***

(a + b).c = ac + bc (distributivité du produit logique sur la somme logique)

ab + c = (a + c).(b + c) (distributivité de la somme logique sur le produit logique)

Les parenthèses imposent une priorité supérieure.

· ***Théorème des constantes***

a + 0 = a *a . 1 = a* (éléments neutres)

a + 1 = 1 *a . 0 = 0*(éléments absorbants)

· ***Complémentation*** : *a + ā = 1* *a . ā = 0*

· ***Involution*** : $\overline{\overline{a}} = a$

· ***Idempotence*** :a + a = a a . a = a

· ***Relation d'absorption***

a + a.b = a a.(a + b) = a

a + ā.b = a + b a.(ā + b) = a.b

· ***Théorème de consensus***

a.b + ā.c + b.c = a.b + ā.c

(a + b).(ā + c).(b + c) = (a + b).(ā + c)

Exercice :

Simplifier les fonctions : $f_1 = abc + \bar{a}bc + a\bar{b}c + ab\bar{c}$

$f_2 = (a + b).c + (a + c).ab + b\bar{c} + a$

I.2. <u>Théorème de De Morgan</u>

Quand on complémente une opération logique complexe, le résultat conduit à substituer l'addition au produit et inversement, ainsi qu'à complémenter les variables.

Le but est d'exprimer les opérateurs ET, OU et NON exclusivement à l'aide d'opérateurs NOR seuls ou NAND seuls.

Premier théorème : $\overline{a + b} = \bar{a}.\bar{b} \rightarrow$ Généralisation : $\overline{\sum_n a_i} = \prod_n \overline{a_i}$

Deuxième théorème : $\overline{a.b} = \bar{a} + \bar{b} \rightarrow$ Généralisation : $\overline{\prod_n a_i} = \sum_n \overline{a_i}$

<u>Expression des opérateurs de base à l'aide des opérateurs complets</u>

- $\bar{a}$ =
- ab =
- a + b =

Les formes normales (ou canoniques)

On appelle **minterme** de *n* variables, un produit logique de ces n variables ou de leurs compléments. Avec *n* variables, on construit 2^n mintermes, c'est-à-dire autant que de combinaisons possibles de *n* éléments prenant deux états. Une ***somme canonique*** est la somme de mintermes.

Un **DEMULTIPLEXEUR** est un système combinatoire qui réalise les 2^n mintermes de n variables.

Exemple : pour 2 variables *a* et *b*, voici les 4 mintermes : ab, .

On appelle **maxterme** de *n* variables, une somme logique de ces dernières (complémentées ou non). De la même manière que pour les mintermes, on construit 2^n maxtermes avec *n* variables. Un ***produit canonique*** est le produit de maxtermes.

Exemple : pour 2 variables *a* et *b*, voici les 4 maxtermes : *a* + *b,* .

➲ La **première forme canonique** (ou 1er théorème de Shannon ou Forme Normale disjonctive FND) d'une expression booléenne est composée d'une somme de mintermes exclusivement. Pour une expression donnée cette forme est unique.

Il faut savoir que f(a, b, c) = a f(1, b, c) + ā f(0, b, c)

Remarque : la somme de tous les mintermes de *n* variables vaut toujours 1 puisqu'il existe toujours un minterme de *n* variables valant 1.

➲ La **seconde forme canonique** (ou 2ème théorème de Shannon ou Forme Normale Conjonctive FNC) d'une expression booléenne est composée d'un produit de maxtermes exclusivement. Pour une expression donnée cette forme est unique.

Il faut savoir que f(a, b, c) = (a + f(0, b, c)).(ā + f(1, b, c)).

Remarque : Le produit de tous les maxtermes de *n* variables vaut toujours 0 puisqu'il existe toujours un maxterme de *n* variables valant 0.

Pour changer de forme canonique on effectue une double complémentation (involution) de l'expression suivie de l'application de l'un des théorèmes de De Morgan.

➲ L'écriture des expressions logique a cet inconvénient d'être assez longue. Chaque minterme parmi les 2^n de *n* variables correspond à un nombre représentant son ordre, c'est pourquoi on préfère parfois utiliser une écriture indiquant la liste classée des numéros des mintermes de la première forme canonique. Cette forme est appelée ***canonique décimale***.

Exemple : $F = abcd + \bar{a}bc\bar{d} + a\bar{b}c\bar{d} + \bar{a}\bar{b}\bar{c}\bar{d}$ peut aussi s'écrire $F(a, b, c, d) = \Sigma\ 0, 6, 10, 15$.

Equivalence entre la table de vérité et les formes canonique

Le but ici est de retrouver l'expression canonique de la fonction connaissant sa table de vérité

- Pour établir l'expression canonique disjonctive de la fonction : il suffit d'effectuer la somme logique des mintermes associées aux états pour lesquels la fonction vaut « 1 ».

- Pour établir l'expression canonique conjonctive de la fonction : il suffit d'effectuer le produit logique des maxtermes associées aux états pour lesquels la fonction vaut « 0 ».

Exemple :

I.3. Simplification algébrique

Il n'existe pas de méthode générale pour simplifier une fonction logique. Néanmoins, deux étapes interviennent souvent dans la simplification.

- L'application du théorème de Morgan pour parvenir à une somme de produits.

- La recherche de variables commune pour parvenir à une simplification.

Exemple :

1- Simplifier $x = \overline{\left(\overline{A} + C\right) . \left(B + \overline{D}\right)}$

2- Réaliser la fonction $y = \overline{A} + \overline{B} + C$ avec une porte NAND et un inverseur.

I.4. Méthode de Karnaugh

Cette méthode permet de simplifier des fonctions pour lesquelles on donne la table de vérité. Elle n'est concrètement utilisable que pour un nombre limité de variables d'entrées. Comme pour la simplification algébrique, la méthode est donc limitée à des fonctions relativement simples.

Chaque combinaison de variables est représentée par une case qui contient la valeur de la fonction. Deux cases sont dites adjacentes quand une seule des variables d'entrée a évolué pour que l'on passe de l'une à l'autre. Graphiquement, on va alors procéder à des regroupements qui vont conduire à des simplifications.

Lors des raisonnements, on procède en disant qu'un 1 correspond à une réponse vraie et un 0 à une réponse fausse.

La table de Karnaugh est un outil graphique qui permet de simplifier de manière méthodique une équation logique. Nous pourrons obtenir ensuite le schéma optimal correspondant au circuit logique combinatoire. Bien que les tables de Karnaugh soient applicables à des problèmes ayant un nombre quelconque de variables d'entrée, en pratique, ils ne sont plus d'une grande utilité quand le nombre de variables dépasse six. En plus, dans cette section, nous n'allons pas aborder de problèmes ayant plus de quatre entrées. Les cas de circuits ayant cinq et six entrées sont des problèmes d'envergure. Au-delà, il est préférable de résoudre les tables de Karnaugh avec un programme informatique.

La construction de la table de Karnaugh

Cette forme de représentation est utilisée pour trouver une expression simplifiée d'une fonction logique. La table de Karnaugh, tout comme la table de vérité, est un instrument qui met en évidence la correspondance entre les entrées logiques et la sortie recherchée. La table de vérité donne la valeur de la sortie X pour chacune des combinaisons des valeurs d'entrée, par contre, la table de Karnaugh organise l'information de manière différente. A chaque ligne de la table de vérité correspond une cellule de la table de Karnaugh. Mais les cellules dans la table de Karnaugh ne sont pas dans le même ordre.

La construction de la table de Karnaugh garanti qu'il n'y a qu'une seule variable qui change entre deux cases voisines. Nous voyons une autre propriété de la table de Karnaugh :

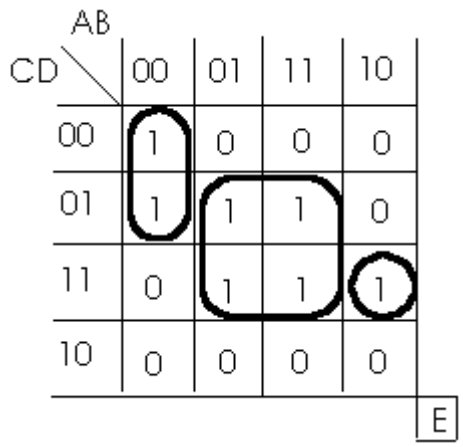
Chaque case à autant de cases adjacentes qu'il y a de variables

Dans le cas d'un système à quatre variables d'entrée, on crée un tableau à 2 x 4 entrées, puis on regroupe les termes adjacents.

Par exemple, soit la table de vérité suivante :

A	B	C	D	E
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

La résolution par Karnaugh donne :



Notez que les lignes 2,3 et les colonnes 2,3 présentent une variable. C'est ainsi que le regroupement du centre s'écrit : $B \cdot D$.

Le regroupement d'en haut à droite représente une simplification moindre : $\overline{A} \cdot \overline{B} \cdot \overline{C}$.

On obtient pour l'expression de la sortie : $E = B \cdot D + \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C \cdot D$

Les conditions indifférentes

Certains circuits logiques peuvent être conçus où pour certaines combinaisons d'entrée ne correspondent aucun état logique particulier pour la sortie. La principale raison est souvent que ces combinaisons d'entrées ne doivent jamais survenir. En d'autres mots, il y a certaines combinaisons des d'entrée pour lesquels il nous importe peu que la sortie soit HAUTE ou BASSE.

C	B	A	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	x
1	0	0	x
1	0	1	1
1	1	0	1
1	1	1	1

Dans cette table aucun état de la sortie Z ne figure pour les combinaisons CBA = 100 et CBA = 011. Au contraire, on a mis un x. Ce x signifie une condition indifférente. Plusieurs raisons peuvent expliquer la présence de conditions indifférentes, la plus courante étant que dans certaines situations ces combinaisons d'entrée ne peuvent jamais survenir; par conséquent, il est inutile de préciser pour elles une valeur de sortie.

En présence de conditions indifférentes, il faut décider quel x de sortie est remplacé par un **0** et lequel est remplacé par un **1**. Le choix est donné en recherchant la façon la plus efficace de grouper les 1 adjacents de la table de Karnaugh. L'objectif est d'obtenir l'expression la plus simple.

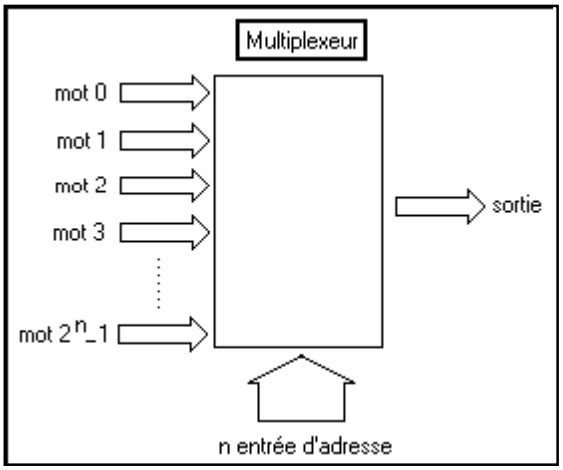
II. FONCTIONS PRINCIPALES DE LA LOGIQUE COMBINATOIRE

Les portes que nous avons décrites précédemment constituent les circuits combinatoires de base. Ces composants utilisés jusqu'à maintenant (ET, OU, NON-ET, XOR, ...) faisaient partie de la catégorie SSI (*Small Scale Integration*). Principalement à cause de leur simplicité, ce sont les premiers circuits intégrés à avoir été réalisés avec succès au tout début de l'ère électronique moderne.

Cependant, on a souvent besoin de travailler avec des fonctions plus évoluées, obtenues à partir d'associations judicieuses de portes élémentaires. Les progrès techniques réalisés en conception de circuits intégrés ont permis de concevoir des circuits un peu plus complexes permettant de réaliser des fonctions plus générales. Ces circuits d'intégration moyenne (MSI – *Medium Scale Integration*) sont présentés dans cette section.

Nous allons décrire quelques-unes de ces fonctions.

II.1. Le multiplexeur.



Un multiplexeur est un système combinatoire qui met sur sa sortie unique la valeur d'une de ses 2^n entrées de données, le numéro de l'entrée sélectionnée étant fourni sur les n entrées de commande ou d'adresse.

Une autre définition est que :

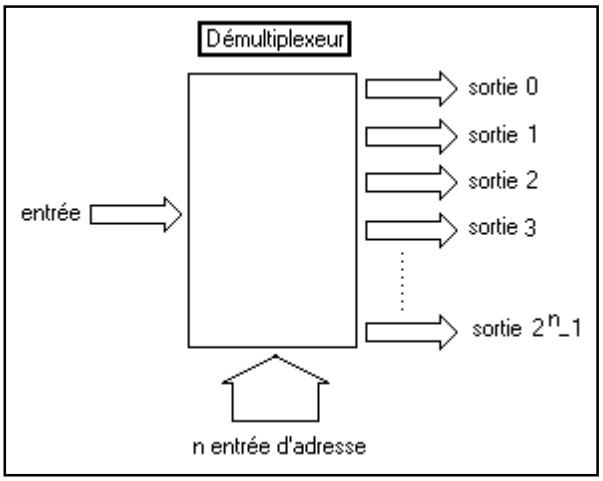
C'est un dispositif comportant 2^n entrées "informations", n entrées "adresses" et une sortie unique. Sa fonction élémentaire est d'assurer l'aiguillage d'une information (à savoir 0 ou 1).

C'est un circuit logique ayant plusieurs entrées de données, mais une seule sortie pour les restituer. La donnée restituée en sortie dépend d'une commande apportée par d'autres entrées (les entrées d'adresse) qui donnent le numéro de canal à transmettre en sortie. Les mots d'entrée comportent autant de bits que la sortie.

Intérêt

- Un multiplexeur joue le rôle d'un commutateur à plusieurs positions qui aiguillerait vers la sortie les informations de n'importe quelle entrée,
- Les entrées d'adressage permettent de connaître à chaque instant l'état d'une entrée de données,
- Par une utilisation cyclique des entrées d'adressage, les données des entrées, qui sont en parallèle à l'entrée du multiplexeur, se retrouvent en série à la sortie.

II.2. Le démultiplexeur.



Ce circuit permet de transmettre un mot logique à un canal dont le numéro sera choisi par l'entrée d'adresse.

C'est un circuit combinatoire à N+1 entrées et 2^N sorties. N entrées, appelées entrées d'adressage, permettent d'envoyer sur l'une des sorties la dernière entrée, appelée l'entrée donnée.

Intérêt

- Un démultiplexeur joue le rôle d'un commutateur à plusieurs positions qui oriente les informations d'entrée vers la sortie choisie,
- Les entrées d'adressage permettent de connaître à chaque instant l'état d'une sortie,
- Par une utilisation cyclique des entrées d'adressage, les données des entrées, qui sont en série à l'entrée du démultiplexeur, se retrouvent en parallèle à la sortie.

II.3. Opérateurs arithmétiques et logiques

.

Un micro-ordinateur utilise le binaire pour faire ses calculs. L'objectif de ce paragraphe est d'étudier la méthode qu'il emploie.

Un calculateur doit pouvoir effectuer toutes les opérations arithmétiques élémentaires (addition, soustraction, multiplication et division). En fait addition et soustraction suffisent, car la multiplication est une addition répétée et la division une soustraction répétée.

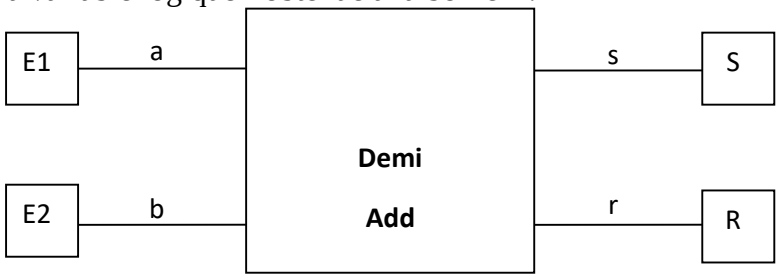
Comme un ordinateur est un calculateur, il ne peut réaliser que les deux opérations de base précitées : l'addition et la soustraction, à l'aide de portes logiques (et, ou, ...), mais chaque opération dure très peu de temps (< 1 ns). Ainsi lorsqu'on lui demande d'additionner 12 + 10, l'ordinateur doit convertir ces nombres en binaire avant d'effectuer cette opération.

Finalement cela revient à faire des additions élémentaires où apparaissent des chiffres qui sont le résultat de la **somme s** et éventuellement des chiffres de **retenue r**.

ATTENTION : La somme Logique est différente de la somme :
Exemple : en logique binaire 1+1=1 OU 1 = 1 tandis que en addition binaire $1_{(2)} + 1_{(2)} = 1\ 0_{(2)}$ (2 en binaire)

a		b		Somme	Retenue
0	+	0	=		
0	+	1	=		
1	+	0	=		
1	+	1	=		

Il faut disposer d'un module permettant de réaliser l'addition de 2 chiffres de 1 bit figurés par les variables logiques a et b correspondant aux entrées E1 et E2 ci-dessous. Le résultat de l'addition est la variable logique s obtenue à la sortie S, une éventuelle retenue étant figurée par la variable logique r obtenue à la borne R.



On peut alors associer une table de vérité et donc une fonction logique pour chacune des 2 sorties :

A	b	s
0	0	
1	0	
0	1	
1	1	

Quelle fonction logique lie **s** à **a** et **b** ?

.....

a	b	r
0	0	
1	0	
0	1	
1	1	

Quelle fonction logique lie **r** à **a** et **b** ?

.....

Additionneur complet (2bits)

Les soustracteurs

III. APPLICATIONS SIMPLES DE LA LOGIQUE COMBINATOIRE

Il faut rappeler qu'un circuit est combinatoire s'il effectue une opération indépendante du temps, c'est-à-dire que son état de sortie à un instant donné ne dépend que de l'état appliqué à ses entrées au même instant.

Outre les multiplexeurs, démultiplexeurs et les opérateurs arithmétiques logiques, nous parlerons ici des aiguilleurs, codeurs, décodeurs et transcodeurs.

III.1. Les aiguilleurs

Pour 2 variables a et b on rencontre régulièrement deux problèmes :

a) On veut obtenir à la sortie S d'un opérateur l'une ou l'autre des variables d'entrée selon l'état de la commande **k**, donc : $\boxed{S = \overline{\overline{ak}.\overline{bk}} = \overline{ak + bk} = ak + b\bar{k}}$. Ce qui conduit au schéma

b) Un signal « a » doit être transmis sur une sortie S1 ou une sortie S2 selon la valeur d'une variable de commande k.
Pour N variables à l'entrée, on utilise un décodeur.

III.2. Codeurs ou encodeurs

C'est un circuit à N entrées dont une seulement est active et qui délivre sur n sorties (en code binaire ou autre) le numéro de l'entrée. Autrement dit c'est un dispositif qui traduit les mots appliqués à l'entrée dans un code choisi. Ainsi par exemple la transposition des chiffres de 0 à 9 s'effectue avec un codeur. Si en effet on porte à 1 (haut) une entrée i (=0 à 9), les sorties A, B, C, D viennent au niveau 1 ou 0 selon le cas.

III.3. Décodeurs

C'est un circuit à n entrées qui permet de sélectionner une sortie parmi N (avec $N \leq 2^n$).

III.4. Transcodeurs

Les transcodeurs sont des circuits qui convertissent les différents codes entre eux. D'une manière générale, nous pouvons définir le transcodage comme étant la représentation d'**un élément de l'ensemble A** par un ou plusieurs éléments pris **dans un ensemble B d'éléments**. Le rapport entre les deux quantités est déterminé par une règle de correspondance.

Etude d'un transcodeur binaire réfléchi / binaire naturel.

Ce type de circuit permet de convertir une position codée en binaire réfléchi (voir codeur de position) en un nombre binaire correspondant à cette position.

Entrée : binaire réfléchi xyzt

Sortie : binaire naturel XYZT

Tables de vérité.

x	y	z	t
0	0	0	0
0	0	0	1
0	0	1	1
0	0	1	0
0	1	1	0
0	1	1	1
0	1	0	1
0	1	0	0
1	1	0	0
1	1	0	1
1	1	1	1
1	1	1	0
1	0	1	0
1	0	1	1
1	0	0	1
1	0	0	0

X	Y	Z	T
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

zt xy	00	01	11	10
00	1	2	3	4
01	8	7	6	5
11	9	10	11	12
10	16	15	14	13

Tableaux de KARNAUGH.

zt / xy	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

zt / xy	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

zt / xy	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	0	0	1	1
10	1	1	0	0

zt / xy	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

Equations :

X = x

Y = $x\ \bar{y} + y\ \bar{x}$ **= x** $\oplus$ **y**

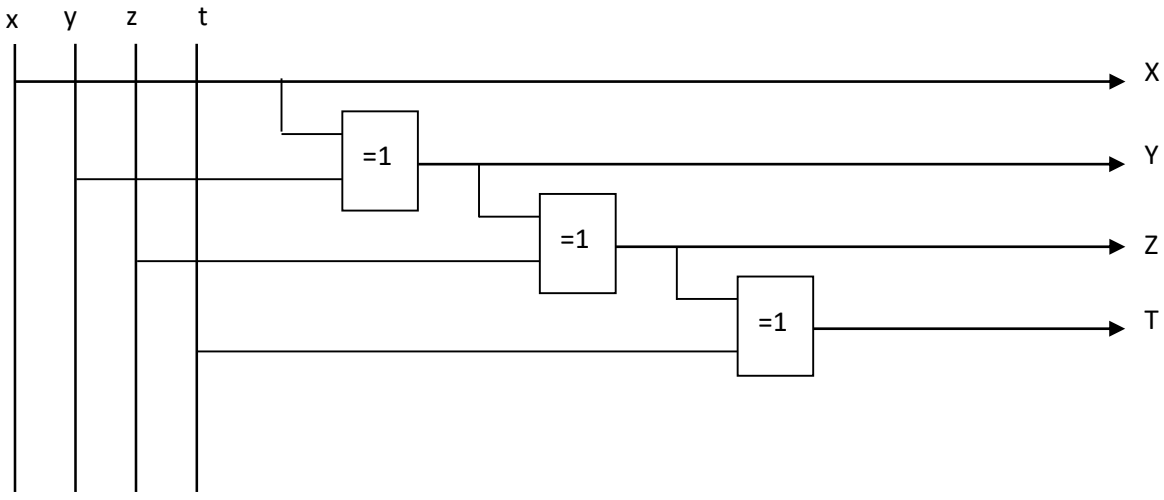
Z = $\bar{x}\ \bar{y}\ z + \bar{x}\ y\ \bar{z} + x\ y\ z + x\ \bar{y}\ \bar{z}$

$= z\ (\bar{x}\ \bar{y} + x\ y) + \bar{z}\ (\bar{x}\ y + x\ \bar{y})$ **= z** $\oplus$ **(x** $\oplus$ **y)**

T = $\bar{x}\ \bar{y}\ \bar{z}\ t + \bar{x}\ \bar{y}\ z\ \bar{t} + \bar{x}\ y\ \bar{z}\ \bar{t} + \bar{x}\ y\ z\ t + x\ y\ \bar{z}\ t + x\ y + z\ \bar{t} + x\ \bar{y}\ \bar{z}\ \bar{t} + x\ \bar{y}\ z\ t$

T = **t** $\oplus$ **(z** $\oplus$ **(x** $\oplus$ **y))**

Logigramme :



Remarque : Nous voyons apparaître une structure répétitive qui permet d'étendre à n bits ce système de transcodage.

EXERCICE N° 1

Prouver les assertions suivantes en partant de l'expression de gauche (***préciser les propriétés et théorèmes utilisés***).

1) $x.y + \bar{x}.z + y.z = x.y + \bar{x}.z$

2) $\bar{x}\ z + x\ \bar{z} + \bar{y}\ z = \bar{x}\ z + x\ \bar{z} + x\ \bar{y}$

3) $\overline{x.z + y.\bar{z}} = \bar{x}.z + \bar{y}.\bar{z}$

EXERCICE N° 2

Construisez le circuit de chacune des fonctions logiques suivantes en utilisant le minimum de portes logique possibles (***simplifiez si nécessaire avant la construction du circuit***)

1) $S_1 = xy + \bar{x}.\bar{y}$

2) $S_2 = \overline{\overline{x \oplus y} \ \oplus \ (z \oplus x)}$

Pour chacune des expressions suivantes, construisez le circuit logique correspondant en recourant à des portes OU, des portes ET et des INVERSEURS.

3) $S_3 = \overline{A.B.\ (C \ \oplus \ D)}$

4) $S4 = \overline{(A + B + \bar{C}.D.\bar{E}) + \bar{B}.C.\bar{D}}$

EXERCICE N° 3

Réduire les équations en utilisant les propriétés de l'algèbre de Boole:

- $A = a + a \cdot b \qquad B = a \cdot (a + b) \qquad C = a + \bar{a} \cdot b$
- $D = (a + b) \cdot (a + \bar{b}) \qquad E = (a + b) \cdot (a + c)$
- $F = (a + b) \cdot (\bar{a} + c) \qquad P = (a + \bar{b}) \cdot (b + \bar{c}) \cdot (c + \bar{a})$
- $Q = (a + b + c) \cdot (a + \bar{b} + c) \cdot (a + \bar{b} + \bar{c})$
- $T = a \cdot b \cdot c + a \cdot \bar{b} \cdot c + a \cdot b \cdot \bar{c} \qquad U = (\bar{a} + b) \cdot (a + b + d) \cdot \bar{d}$
- $V = (a + b) \cdot (a + c) + (b + c) \cdot (b + a) + (c + a) \cdot (c + b)$

EXERCICE N° 4

Déterminer les fonctions représentées par ces tableaux de Karnaugh :

P		a b			
		0 0	0 1	1 1	1 0
c d	0 0	1	0	0	1
	0 1	1	1	1	1
	1 1	1	1	0	0
	1 0	0	0	0	0

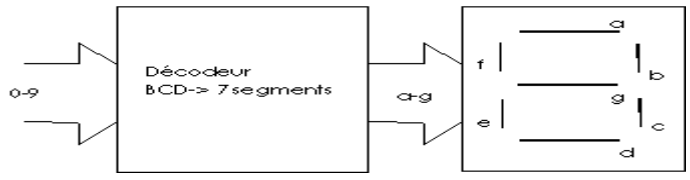
Q		a b			
		0 0	0 1	1 1	1 0
c d	0 0	0	0	1	0
	0 1	1	0	1	1
	1 1	1	1	1	1
	1 0	0	0	1	0

R		a b			
		0 0	0 1	1 1	1 0
c d	0 0	0	1	1	0
	0 1	1	0	0	1
	1 1	0	0	0	0
	1 0	0	1	1	0

S		a b c							
		000	001	011	010	110	111	101	100
d e	0 0	0	1	1	0	0	1	1	0
	0 1	0	1	1	0	0	1	1	0
	1 1	0	1	1	0	0	1	1	0
	1 0	1	1	1	0	0	1	1	0

EXERCICE N° 5

1- Remplissez la table de vérité d'un décodeur à 7 segments :



2- En utilisant Karnaugh, exprimez ce décodeur par une fonction logique.

3- Représentez ce décodeur à l'aide des portes logiques standard.

EXERCICE N° 6 Problème de commande d'un distributeur de boissons

3 boutons commandent 3 électrovannes branchées à 3 cuves contenant des liquides (eau, cassis, menthe). Le distributeur permet d'obtenir 3 boissons :

- E : eau

- C : cassis

- M : menthe

Si l'on veut un mélange, on appuie sur 2 boutons simultanément (e et c) ou (e et m) et on introduit une pièce (p).

Tous les autres mélanges sont interdits.

L'eau est gratuite. La commande de cassis ou menthe vous donne ces produits dilués.

Pour toute fausse manœuvre (mélange interdit ou eau seule) la pièce est restituée.

- P: pièce restituée

Exprimer E, C, M et P en fonction de e, c, m et p.

EXERCICE N° 7

Un nombre binaire de 4 bits est noté $A_3A_2A_1A_0$, où A_3, A_2, A_1 et A_0 représentent chacun des bits, A_0 étant le bit de plus faible poids. Concevez un circuit logique qui donne une sortie HAUTE quand le nombre binaire est strictement supérieur à 0011 et inférieur à 1001.

EXERCICE N° 8

Exprimer U et V en fonction de A,B,C et D, le plus simplement possible

A B C D	U
0 0 0 0	0
0 0 0 1	1
0 0 1 0	1
0 0 1 1	0
0 1 0 0	0
0 1 0 1	1
0 1 1 0	0
0 1 1 1	0
1 x x x	0

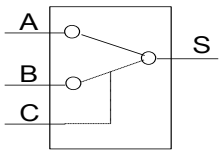
A B C D	V
0 0 0 0	1
0 0 0 1	1
0 0 1 0	0
0 0 1 1	0
0 1 0 0	1
0 1 0 1	1
0 1 1 0	0
0 1 1 1	0
1 x x x	1

EXERCICE N° 9

Soit un circuit multiplieur qui accepte en entrée deux nombres binaires de 2 bits x_1x_0 et y_1y_0 et qui fournit en sortie le nombre binaire $S_3S_2S_1S_0$ égal au produit arithmétique des deux nombres d'entrée. Créez le circuit logique du multiplicateur. Pour concevoir le circuit, on pourra l'envisager comme ayant quatre entrées et quatre sorties.

EXERCICE N° 10

Proposer un schéma structurel à l'aide de ET NON à deux entrées pour réaliser la fonction:

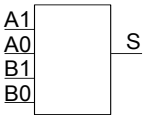


si C=1 alors S=A

si C=0 alors S=B

EXERCICE N° 11

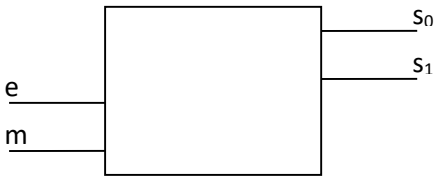
Proposer un schéma structurel à l'aide de ET NON à deux entrées pour réaliser la fonction:



$$S = 1 \Leftrightarrow (A1A0)_2 < (B1B0)_2$$

EXERCICE N° 12

C'est la fonction inverse du multiplexeur :



m = 0 : s_0 = e s_1 = 0

m = 1 : s_1 = e s_0 = 0

1- Etablir la table de vérité.

2- En déduire l'équation booléenne des sorties.